

Пояснювальна записка

до магістерської дисертації

на тему: ***Програмний засіб розподілу навантаження
порталу служби підтримки на базі моделі узагальнення
тексту***

виконав студент групи ІІІ-43мп

Цуканова Марина Сергіївна

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ.....	4
ВСТУП	5
1 АНАЛІЗ МЕТОДІВ ДЛЯ РОЗПОДІЛУ НАВАНТАЖЕННЯ СЛУЖБИ ПІДТРИМКИ	7
1.1 Опис предметної області.....	7
1.2 Аналіз існуючих рішень.....	8
1.3 Постановка задачі	11
2 РОЗРОБКА ІЄРАРХІЧНОЇ МОДЕЛІ КЛАСИФІКАЦІЇ ТА АЛГОРИТМУ РОЗПОДІЛУ НАВАНТАЖЕННЯ	13
2.1 Підготовка та аналіз даних для навчання моделей класифікації.....	13
2.2 Розробка бінарного класифікатора для фільтрації запитів	16
2.3 Оцінка ефективності бінарного класифікатора	18
2.4 Розробка мультикласового класифікатора для визначення типу ІТ-завдання.....	19
2.5 Оцінка ефективності мультикласового класифікатора.....	20
2.6 Розробка алгоритму автоматичного розподілу завдань	23
3 РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	27
3.1 Опис вимог до програмного забезпечення.....	27
3.1.1 Опис функціональним вимог.....	27
3.1.2 Опис нефункціональних вимог	29

3.2	Вибір технологій.....	30
3.3	Розробка архітектури.....	31
3.4	Інтерфейс користувача	34
3.5	Інструкції по розгортанню ПЗ.....	48
3.6	Експериментальні дослідження	50
3.7	Оцінка ефективності запропонованого рішення	52
4	МАРКЕТИНГОВИЙ АНАЛІЗ СТАРТАП-ПРОЄКТУ	55
4.1	Опис ідеї проєкту.....	55
4.2	Технологічний аудит ідеї проєкту	56
4.3	Аналіз ринкових можливостей запуску стартап-проєкту	57
4.4	Розроблення ринкової стратегії проєкту	63
4.5	Розроблення маркетингової програми програми стартап-проєкту	66
	ВИСНОВКИ.....	70
	СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	72
	ДОДАТКИ.....	74
	ДОДАТОК А.....	75
	ДОДАТОК Б	76
	ДОДАТОК В.....	77
	ДОДАТОК Г	78
	ДОДАТОК Д.....	101
	ДОДАТОК Е	103

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

БД – база даних;

API (application programming interface) – програмний інтерфейс;

ІТ – інформаційні технології;

HTTP (Hypertext Transfer Protocol) – протокол передачі даних;

NLP (Natural Language Processing) – обробка природної мови;

BERT – методика машинного навчання для попереднього тренування обробки природної мови.

ВСТУП

В сучасному бізнес-середовищі ефективність роботи служби підтримки інформаційних технологій (ІТ) є критично важливою для забезпечення стабільності операційних процесів та задоволеності користувачів. Традиційні підходи до обробки заявок, що часто передбачають ручний аналіз та розподіл завдань менеджером, стикаються з проблемами масштабованості, затримками у реагуванні та неоптимальним використанням ресурсів. Зростаючий обсяг запитів та підвищення вимог до швидкості обслуговування зумовлюють нагальну потребу в автоматизації цих процесів. Робота є актуальною, оскільки вирішує сучасні проблеми у сфері служб ІТ підтримки, спрямовані на підвищення швидкості реагування та якості обслуговування. Вона відповідає світовим тенденціям до впровадження інтелектуальних систем та автоматизації обробки інформації за допомогою методів обробки природної мови (NLP) та машинного навчання (ML), що є важливим для оптимізації роботи сучасних організацій.

Сучасні дослідження та комерційні рішення активно використовують NLP та ML для автоматизації завдань ІТ-підтримки. Спостерігається перехід від класичних методів машинного навчання до більш потужних архітектур глибокого навчання, зокрема трансформерів (BERT, DistilBERT, LLM), які демонструють високу ефективність у класифікації текстових запитів та розпізнаванні намірів користувачів. Провідні ITSM-платформи, такі як ServiceNow, Jira Service Management, Zendesk, інтегрують подібні AI-рішення для автоматичного визначення категорії, пріоритету та навіть для пропозиції варіантів вирішення. Також розвиваються підходи до автоматичного розподілу завдань між виконавцями з урахуванням типу проблеми та навантаження.

Однак, незважаючи на значний прогрес, існують невирішені проблеми та прогалини знань. По-перше, більшість існуючих моделей та досліджень

фокусуються на англійській мові, тоді як відсутні публічні, донавчені моделі, спеціалізовані саме для класифікації українськомовних ІТ-заявок. По-друге, стандартні мультикласові класифікатори погано справляються з обробкою нерелевантних або нетипових запитів (Out-of-Distribution), що є критичним для надійної роботи системи підтримки. По-третє, ефективна інтеграція результатів класифікації з алгоритмами розподілу навантаження потребує подальших досліджень та адаптації до специфіки конкретних команд підтримки.

Метою даної магістерської дисертації є підвищення ефективності роботи служби підтримки ІТ відділу шляхом розробки програмного засобу, що автоматизує процеси класифікації та розподілу текстових заявок на основі адаптованих моделей аналізу тексту. Досягнення мети передбачає часткову автоматизацію обробки запитів для пришвидшення їх опрацювання. Критерієм ефективності виступає швидкість опрацювання заявок. Розроблений програмний засіб призначений для використання у внутрішніх службах ІТ-підтримки компаній.

Об'єктом дослідження є програмне забезпечення розподілу навантаження порталу служби підтримки.

Предметом дослідження є методи та засоби автоматизації процесу обробки заявок та визначення терміновості заявки з її тексту.

Наукова новизна одержаних результатів полягає у набутті подальшого розвитку застосування NLP-моделей для автоматизації процесів ІТ-підтримки шляхом інтеграції ієрархічної класифікації з алгоритмом розподілу завдань, що враховує компетенції та навантаження виконавців.

1 АНАЛІЗ МЕТОДІВ ДЛЯ РОЗПОДІЛУ НАВАНТАЖЕННЯ СЛУЖБИ ПІДТРИМКИ

1.1 Опис предметної області

Сучасні організації значною мірою залежать від стабільної роботи інформаційних технологій (ІТ), що робить служби ІТ-підтримки критично важливим елементом їхньої інфраструктури. Основне завдання таких служб – оперативно вирішувати технічні проблеми та відповідати на запити користувачів (працівників компанії або клієнтів), забезпечуючи безперебійність бізнес-процесів.

Для користувачів ключовими факторами якості ІТ-підтримки є швидкий час реагування на запит, наявність кваліфікованого персоналу та зручні інструменти для взаємодії зі службою. Спостерігається тенденція до самообслуговування: значна частина користувачів (близько 67%) надає перевагу самостійному пошуку рішень через портали підтримки (наприклад, бази знань, FAQ), аніж прямому спілкуванню з оператором [1]. Такі портали є зручним та економічно вигідним рішенням, дозволяючи користувачам швидко знаходити відповіді на поширені запитання без залучення спеціалістів.

Однак, у випадках, коли самообслуговування недостатньо або проблема є унікальною, користувачі звертаються до служби підтримки напрямку, часто через створення текстових заявок на порталі. Стандартний процес обробки таких заявок зазвичай включає кілька ручних етапів:

- користувач описує проблему у текстовій формі;
- менеджер служби підтримки отримує заявку
- менеджер читає та аналізує текст заявки, щоб визначити її тип (наприклад, проблема з мережею, запит на ПЗ, проблема з доступом);
- менеджер призначає відповідального виконавця (спеціаліста відповідної кваліфікації).

Цей ручний процес має суттєві недоліки [2]. По-перше, він потребує значного часу та ресурсів менеджера, особливо при великому потоці заявок. По-друге, він може призводити до затримок у реагуванні, що негативно впливає на задоволеність користувачів та продуктивність роботи. Водночас аналіз показує, що значна частина запитів до IT-підтримки є типовими та повторюваними, що створює передумови для автоматизації їх обробки.

Таким чином, існує нагальна потреба в удосконаленні процесів роботи служб IT-підтримки шляхом впровадження інтелектуальних програмних засобів. Автоматизація етапів аналізу тексту заявки, визначення її типу та призначення виконавця може суттєво зменшити час обробки запитів, знизити навантаження на менеджерів та підвищити загальну ефективність роботи служби підтримки. Це, своєю чергою, сприятиме покращенню досвіду користувачів та стабільності IT-інфраструктури організації.

1.2 Аналіз існуючих рішень

Автоматизація процесів у службах IT-підтримки є актуальною задачею, спрямованою на підвищення ефективності обслуговування та скорочення часу реакції на запити користувачів. Сучасні підходи активно використовують методи обробки природної мови (NLP) та машинного навчання (ML) для аналізу текстових заявок. Основні цілі – автоматична класифікація запитів за типом проблеми, визначення їх пріоритетності та оптимізація розподілу завдань між виконавцями. Аналіз існуючих рішень та технологій дозволяє виділити кілька ключових напрямків:

- рішення на основі класичних методів ML та NLP;
- застосування глибокого навчання та трансформерних архітектур.

Історично перші спроби автоматизації класифікації заявок базувалися на традиційних алгоритмах машинного навчання, таких як Support Vector Machines (SVM), Naive Bayes, або логістична регресія, у поєднанні з

класичними техніками NLP для представлення тексту (TF-IDF, Bag-of-Words). Ці методи застосовувалися для базового сортування тікетів за категоріями або визначення терміновості на основі наявності певних ключових слів [3]. Подібні підходи могли бути реалізовані у ранніх версіях ITSM-систем (IT Service Management), таких як BMC Remedy або кастомних рішеннях, де основна увага приділялася створенню правил маршрутизації.

До переваг класичних методів ML та NLP можна віднести відносну простоту реалізації, інтерпретованість результатів та менші вимоги до обсягу даних для навчання.

До недоліків класичних методів ML та NLP можна віднести низьку здатність до розуміння семантики та контексту тексту заявки, що призводить до помилок при варіативності формулювань. А також високу чутливість до якості ручного створення ознак (feature engineering), обмежену ефективність при роботі з великою кількістю категорій або складними, багатоаспектними запитами.

З розвитком глибокого навчання (DL) з'явилися значно потужніші інструменти для аналізу тексту. Архітектури, такі як рекурентні нейронні мережі (LSTM) та, особливо, трансформери (BERT, RoBERTa, T5, GPT та інші Large Language Models – LLM), демонструють значно вищу ефективність у задачах розуміння природної мови (NLU). Провідні ITSM-платформи, такі як ServiceNow (з модулем Predictive Intelligence), Jira Service Management (з інтеграціями Atlassian Intelligence) та CRM-системи з функціями підтримки (Zendesk AI, Freshdesk Freddy AI) активно впроваджують ці технології.

Вони використовуються для автоматичної класифікації заявок, визначення намірів користувачів, виявлення ключових сутностей, визначення пріоритету, а іноді й для генерації коротких узагальнень проблеми або пропозиції варіантів рішень. Дослідження також показують потенціал

використання LLM для покращення роботи розмовних агентів (чат-ботів) у службах підтримки [4].

До переваг застосування глибокого навчання та трансформерних архітектур можна віднести високу точність у розумінні контексту, семантики та нюансів природної мови, здатність обробляти великі обсяги неструктурованих даних без необхідності ручного створення ознак та можливість реалізації складних функцій (визначення емоцій, терміновості, генерація тексту).

До недоліків застосування глибокого навчання відносять високі вимоги до обчислювальних ресурсів для навчання та донавчання (fine-tuning), необхідність великих та якісних наборів даних, які часто є специфічними для кожної організації, складність інтерпретації рішень моделі. А також стандартні моделі класифікації, навчені на обмеженій кількості класів, погано справляються з нетиповими або нерелевантними запитами (Out-of-Distribution, OOD), часто присвоюючи їм неправильну категорію з високою впевненістю.

Незважаючи на значний прогрес у розробці загальних мовних моделей, існує помітна прогалина у наявності публічних, донавчених (fine-tuned) моделей, спеціалізованих саме на класифікації ІТ-заявок українською мовою. Аналіз літератури показує, що більшість досліджень та готових рішень фокусуються або на англійській мові, або на інших доменах (наприклад, загальний аналіз настрою, медицина, фінанси).

Хоча існують потужні багатомовні моделі (наприклад, distilbert-base-multilingual-cased, xlm-roberta-base), які розуміють українську мову, вони не мають специфічних знань про термінологію та типові проблеми в ІТ-сфері.

Як наслідок, використання загальних моделей "як є" для класифікації ІТ-заявок може призводити до нижчої точності порівняно з моделями, донавченими на релевантних даних. Відсутність готових наборів даних та

бенчмарків для українськомовних ІТ-заявок ускладнює порівняння ефективності різних підходів та розробку спеціалізованих рішень.

Окремим, але тісно пов'язаним завданням є автоматизація розподілу завдань між співробітниками служби підтримки. Сучасні ITSM-платформи (ServiceNow, Jira Service Management, Zendesk) пропонують різні механізми для цього: від простих правил (round-robin, на основі поточної завантаженості) до більш інтелектуальних систем, які намагаються врахувати тип завдання (визначений на попередньому етапі класифікації) та навички/спеціалізацію виконавця. Дослідження також розглядають симуляцію та оптимізацію розподілу завдань у схожих контекстах, наприклад, на виробництві [5].

До переваг використання таких систем відноситься зменшення ручного навантаження на менеджерів та потенціал для більш рівномірного та ефективного розподілу роботи між спеціалістами.

До недоліків цього методу відноситься складність формалізації та підтримки актуальності профілів навичок співробітників, пряма залежність якості розподілу від точності попередньої класифікації заявки, а також некоректно класифікована заявка буде направлена не тому спеціалісту, що призведе до затримок у вирішенні запиту.

1.3 Постановка задачі

Метою роботи є підвищення ефективності роботи служби підтримки ІТ відділу за допомогою використання алгоритму, що дозволяє обробляти запити до служби підтримки з більшою швидкістю, завдяки частковій автоматизації процесів обслуговування, шляхом використання алгоритмів аналізу тексту. Для досягнення мети необхідно вирішити наступні задачі:

- проаналізувати існуючі підходи та програмних засобів для автоматизації обробки запитів в ІТ-службах підтримки;

- розробити ієрархічну модель класифікації текстових запитів, що фільтрує релевантні запити та визначає конкретний тип ІТ-проблеми;
- розробити алгоритм автоматичного розподілу заявок між співробітниками на основі визначеного типу завдання;
- реалізувати програмний засіб у вигляді вебпорталу, що інтегрує розроблену ієрархічну модель та алгоритм розподілу завдань;
- оцінити ефективність запропонованого рішення.

Висновки до розділу

У першому розділі було проведено аналіз предметної області автоматизації процесів служби ІТ-підтримки, розглянуто типові проблеми ручної обробки текстових заявок та визначено ключові потреби користувачів у швидкому та ефективному вирішенні ІТ-проблем.

В результаті проведеного аналізу виявлено потребу у розробці комплексного рішення, яке б поєднувало точну класифікацію українськомовних ІТ-заявок зі здатністю відфільтровувати нерелевантні запити та інтегрувалося б з механізмом автоматичного розподілу завдань. На основі цього сформульовано постановку задачі дослідження, визначено мету роботи та окреслено конкретні завдання розробки програмного засобу, що включають створення ієрархічної моделі класифікації та алгоритму розподілу навантаження.

2 РОЗРОБКА ІЄРАРХІЧНОЇ МОДЕЛІ КЛАСИФІКАЦІЇ ТА АЛГОРИТМУ РОЗПОДІЛУ НАВАНТАЖЕННЯ

2.1 Підготовка та аналіз даних для навчання моделей класифікації

Ефективність моделей машинного навчання, особливо у задачах обробки природної мови, значною мірою залежить від якості та репрезентативності даних, що використовуються для їх навчання. Тому першим кроком у розробці ієрархічної системи класифікації ІТ-заявок була ретельна підготовка та аналіз вихідних даних.

Для навчання було використано два основні типи даних – ІТ-заявки, отримані як корпус текстових запитів, що надходили до служби ІТ-підтримки компанії та “НЕ-ІТ” запити, що були зібрані у додатковий корпус нерелевантних текстових повідомлень.

Початковий набір даних з ІТ заявок містив 2450 записів, розподілених за 7 категоріями:

- Password Issues;
- Security Incidents;
- File and Storage management;
- Application management;
- Account management;
- Software management;
- Network management.

Ці дані було розділено на тренувальний та тестовий датасет у відношенні 80% до 20%. Кожна тренувальна категорія заявок мала 280 записів своєї категорії. Кожна тестувальна категорія заявок містила 70 запитів. Цей датасет було використано для дотренування моделі на базі DistilBERT.

Для навчання бінарного класифікатора, який має відрізнити релевантні ІТ-заявки від інших, було зібрано додатковий корпус нерелевантних текстових

повідомлень. Цей набір включав 500 прикладів, таких як запити до HR, загальні фрази, новини. Ці дані було зібрано з вручну та з публічних корпусів.

Спершу було проведено очищення даних, що складалось з видалення дублікатів, усунення витоку даних та обробки відсутніх значень.

З початкового набору ІТ-заявок було видалено повністю ідентичних записів для уникнення зміщення моделі. Унікальних ІТ-заявок залишилося 1426 запитів. Було проведено перевірку на наявність однакових записів між тренувальним та тестовим наборами даних. Виявлено 190 спільних записів, які було видалено з тестового набору для забезпечення валідності оцінки моделі. Видалено 5 рядків, що мали відсутні значення у тексті заявки або її мітці.

Для зменшення варіативності та уніфікації термінології було застосовано доменно-специфічну нормалізацію тексту. Було створено словник відповідностей, який замінював поширені скорочення, жаргонізми та можливі одруки на їх канонічні форми. Цю операцію було застосовано до всіх текстових заявок (як ІТ, так і Не-ІТ).

Для збільшення обсягу тренувальних даних для мультикласового класифікатора було застосовано метод зворотного перекладу (back-translation [6]). Записи тренувального датасету було перекладено англійською мовою, а потім перекладено українською мовою. Для перекладу тексту заявок було використано моделі Helsinki NLP. Таким чином тренувальний датасет було збільшено вдвічі.

На основі підготовлених даних було сформовано два основні набори для навчання моделей.

Для навчання бінарного класифікатора було об'єднано 500 унікальних ІТ заявок та 500 нерелевантних запитів. Загальний розмір набору склад 1000 записів. Цей набір було розподілено на тренувальний (80%) та валідаційний

(20%) датасет за допомогою стратифікованого розбиття для збереження пропорцій класів.

Для донавчання мультикласового класифікатора було використано 1426 унікальних заявок, кожна з яких мала одну з 7 міток класів. Для оцінки моделі використовувався окремий, попередньо очищений тестовий набір даних розміром 300 записів.

На рисунках 2.1 та 2.2 наведено розподіл класів тренувального та валідаційного датасетів після очищення, нормалізації та аугментації даних.

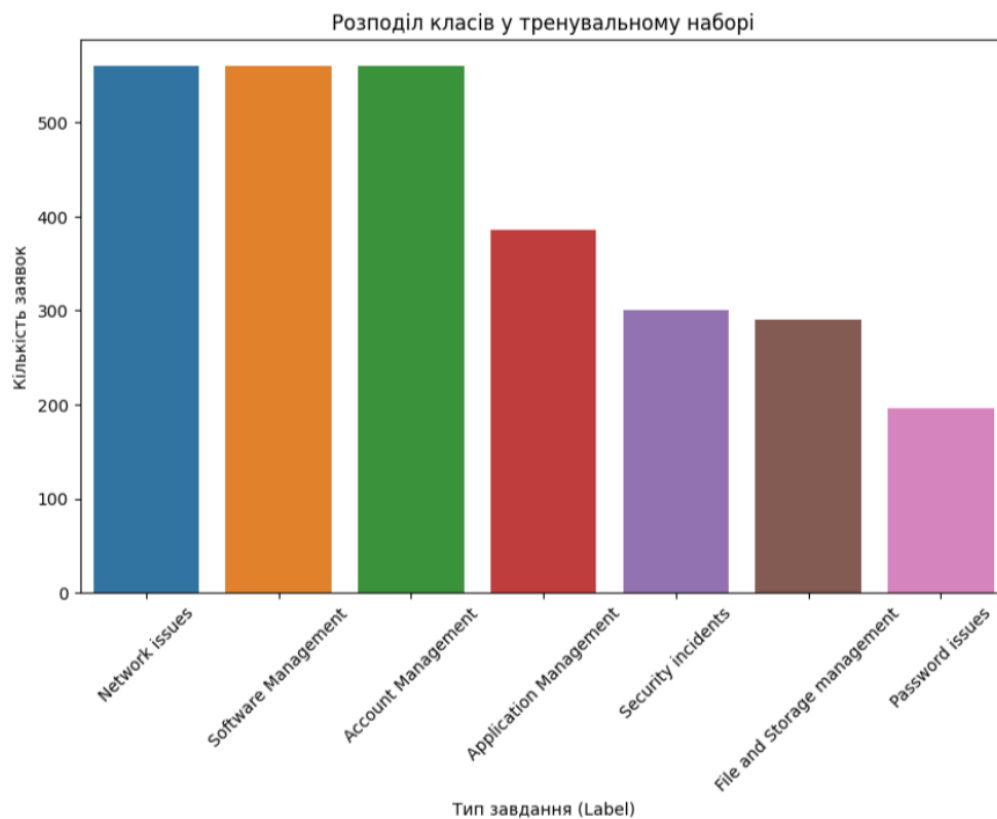


Рисунок 2.1 – Розподіл класів у тренувальному датасеті

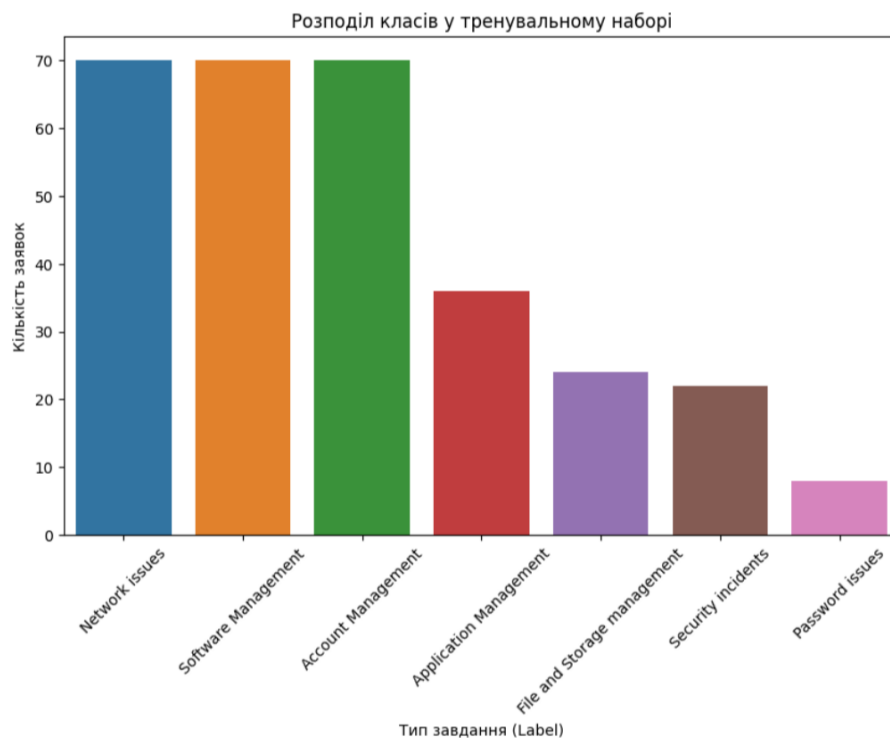


Рисунок 2.2 – Розподіл класів у валідаційному датасеті

Аналіз довжини текстових заявок показав, що середня довжина становить 230 символів, з мінімальною довжиною 51 та максимальною 371. Це свідчить про відсутність аномально коротких чи довгих записів. Розподіл класів у фінальних наборах даних був врахований при виборі метрик оцінки моделей (зокрема, F1 Macro).

Таким чином, було підготовлено два набори даних, очищених, нормалізованих та структурованих для подальшого навчання бінарної та мультикласової моделей класифікації в рамках розроблюваної ієрархічної системи.

2.2 Розробка бінарного класифікатора для фільтрації запитів

Для реалізації першого рівня ієрархічної системи класифікації було розроблено бінарний класифікатор, основне завдання якого відрізнити релевантні IT-заявки від інших ("Не-IT") запитів. Це дозволяє відфільтрувати

нецільовий "шум" перед тим, як запит буде передано на подальшу обробку мультикласовому класифікатору.

В якості базової архітектури було обрано модель DistilBERT (distilbert-base-multilingual-cased). Цей вибір обґрунтований кількома факторами. По перше, модель підтримує українську мову, що є ключовою вимогою. По друге, DistilBERT є полегшеною версією BERT, що забезпечує хороший баланс між точністю та обчислювальною ефективністю (швидкістю навчання та інференсу). По третє, на відміну від моделей, попередньо навчених на специфічних завданнях (напр., аналіз настрою), ця модель має загальне розуміння мови, що є кращою відправною точкою для донавчання на новій бінарній задачі.

Донавчання моделі здійснювалося на підготовленому бінарному наборі даних (описаному в підрозділі 2.1), що містив 800 тренувальних прикладів класів "IT" (мітка 1) та "Не-IT" (мітка 0). Використовувався фреймворк transformers від Hugging Face. Було завантажено переднавчену модель distilbert-base-multilingual-cased та модифіковано для двох вихідних класів.

Для процесу донавчання було використано наступні гіперпараметри та налаштування Trainer:

- розмір батчу (batch size): 16;
- кількість епох навчання: 15;
- оптимізатор: Adam;
- функція втрат: CrossEntropyLoss.

Для запобігання перенавчанню та автоматичного вибору найкращої версії моделі було використано механізм ранньої зупинки (Early Stopping). Тренування зупинялося автоматично, якщо метрика eval_f1_macro на валідаційному наборі не покращувалася протягом 2 епох поспіль. Також було активовано опцію load_best_model_at_end=True, щоб після завершення

тренування в пам'яті залишилася модель з найкращими показниками на валідаційному наборі.

2.3 Оцінка ефективності бінарного класифікатора

Після завершення процесу донавчання було проведено оцінку ефективності розробленого бінарного класифікатора. Оцінка здійснювалася на валідаційному наборі даних, що містив 200 записів і не використовувався безпосередньо під час тренування параметрів моделі.

Для оцінки якості класифікації було обрано метрики Ассюрасу (загальна частка правильно класифікованих запитів) та F1-score (усереднене значення F1-score для обох класів ("ІТ" та "Не-ІТ") без урахування їх розміру).

Модель досягла наступних показників на валідаційному наборі:

- точність (ассюрасу): 0.96;
- f1-score (macro): 0.9714.

В ході донавчання моделі відбувся EarlyStopping. Найкраща модель була отримана у другій епосі тренування. На рисунку 2.3 зображено зміни в показниках моделі. На рисунку 2.4 графічно зображено зміни у показниках loss та f1(macro).

5/15] [895/2685 02:08 < 04:17, 6.1

Epoch	Training Loss	Validation Loss	Accuracy	F1 Weighted	F1 Macro
1	0.804400	0.175531	0.953333	0.953340	0.966254
2	0.189900	0.132083	0.960000	0.959783	0.971412
3	0.077700	0.210837	0.956667	0.956544	0.968216
4	0.032300	0.273123	0.950000	0.950085	0.962963
5	0.012000	0.251272	0.956667	0.956721	0.968325

Рисунок 2.3 – Зміни в показниках моделі

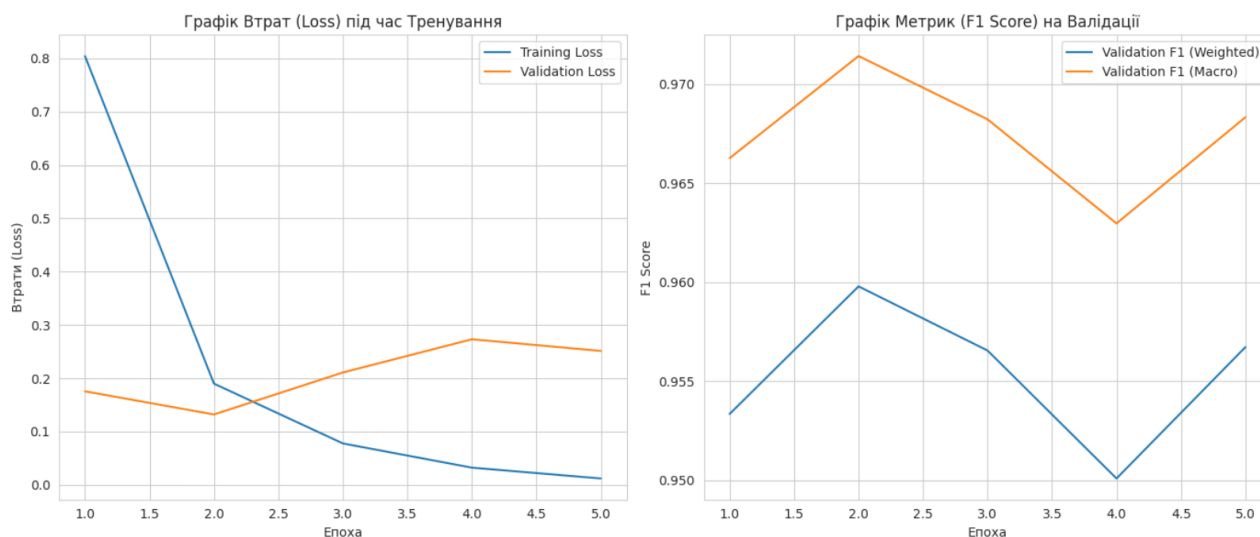


Рисунок 2.4 – Зміни в показниках loss та f1-score

Як видно з отриманих метрик та матриці плутанини, розроблений бінарний класифікатор демонструє дуже високу ефективність у розрізненні ІТ-релевантних та нерелевантних запитів. Значення F1 Macro близьке до 0.97, свідчить про те, що модель майже безпомилково ідентифікує обидва класи.

Таким чином, розроблений бінарний класифікатор є надійним інструментом для попередньої фільтрації запитів і може ефективно використовуватися як перший етап в ієрархічній системі класифікації.

2.4 Розробка мультикласового класифікатора для визначення типу ІТ-завдання

Другим компонентом ієрархічної системи є мультикласовий класифікатор. Його завдання приймати на вхід запити, які були визнані бінарним класифікатором як релевантні ІТ-завдання (клас 1), та визначати їх конкретний тип відповідно до 7 заздалегідь визначених категорій: Software management, Network management, Account management, Application management, File and Storage management, Password Issues, Security Incidents.

Для забезпечення консистентності підходів та можливості порівняння результатів, для мультикласового класифікатора було обрано ту саму базову

архітектуру, що й для бінарного – DistilBERT (distilbert-base-multilingual-cased). Ця модель довела свою ефективність у розумінні української мови на попередньому етапі та є достатньо потужною для вирішення задачі класифікації на 7 класів, зберігаючи при цьому відносну обчислювальну ефективність.

Донавчання моделі здійснювалося на спеціалізованому наборі даних ІТ-заявок (описаному в підрозділі 2.1), що містив 2842 прикладів, розмічених відповідними 7 категоріями. Було завантажено перед-навчену модель distilbert-base-multilingual-cased та адаптовано для 7 вихідних класів.

Для процесу донавчання було використано наступні гіперпараметри та налаштування Trainer:

- Розмір батчу (batch size): 16;
- Кількість епох навчання: 15;
- Оптимізатор: Adam;
- Функція втрат: CrossEntropyLoss.

Також було застосовано механізм ранньої зупинки (Early Stopping) з `patience = 2` для метрики `eval_f1_macro` та активовано опцію `load_best_model_at_end=True` для автоматичного збереження найкращої версії моделі за результатами валідації на кожній епосі.

2.5 Оцінка ефективності мультикласового класифікатора

Ефективність розробленого мультикласового класифікатора оцінювалася на незалежному тестовому наборі даних. Цей набір складався з 300 ІТ-заявок, які не використовувалися під час навчання і були попередньо очищені від записів, що просочилися з тренувального набору (як описано в підрозділі 2.1). Важливо зазначити, що розподіл класів у цьому тестовому наборі був незбалансованим, що імітує реальні умови надходження заявок.

Для комплексної оцінки якості мультикласової класифікації було обрано метрики Accuracy (загальна частка правильно класифікованих ІТ-заявок), F1-score (Weighted Average) (усереднене значення F1-score для всіх 7 класів, зважене за кількістю прикладів кожного класу) та F1-score (Macro Average) (усереднене значення F1-score для всіх 7 класів без урахування їх розміру). Метрика F1-score (macro) є ключовою для оцінки здатності моделі добре класифікувати всі типи завдань, включаючи ті, що рідко зустрічаються в тестовому наборі.

Після тренування (яке було зупинено на 12 епосі з 15) найкраща версія моделі продемонструвала наступні показники на тестовому наборі:

- Accuracy: 0.966;
- F1 Weighted: 0.966;
- F1 Macro: 0.974.

На рисунку 2.5 зображено зміни показників loss та f1-score протягом тренувальних епох. Матриця плутанини (рисунок 2.6) візуалізує, між якими класами модель найчастіше допускала помилки. Деталізовані результати (precision, recall, f1-score) для кожної з 7 категорій наведено у звіті класифікації (рисунок 2.7)

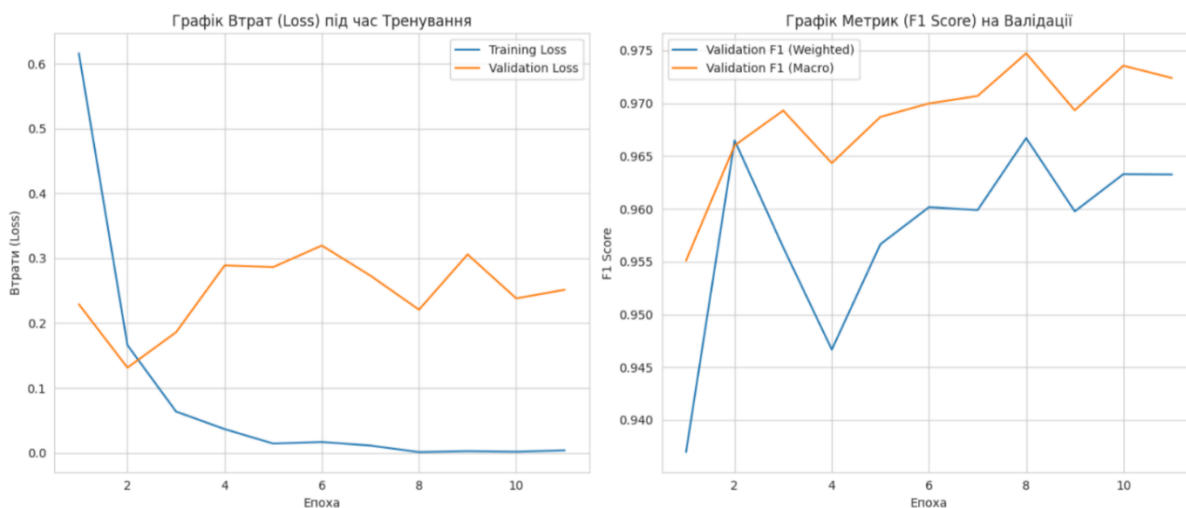


Рисунок 2.5 – Зміни у метриках loss та f1-score протягом епох

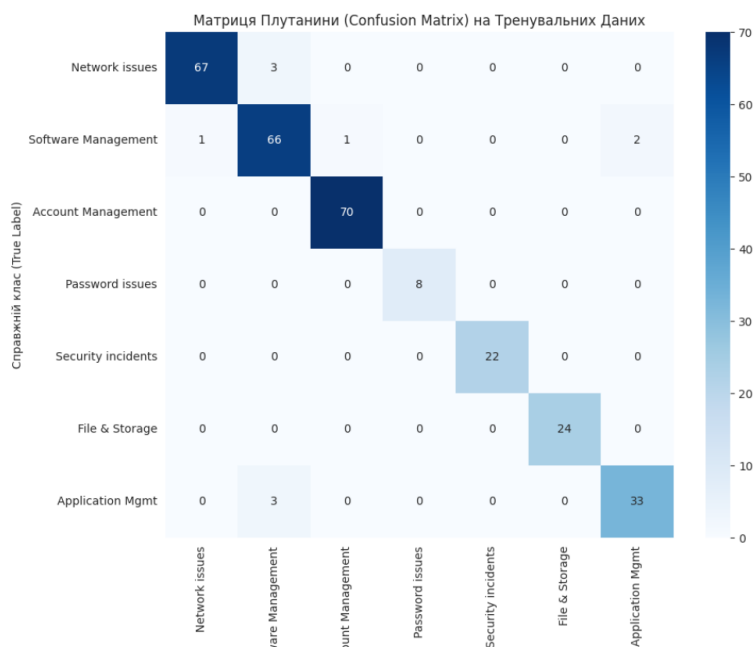


Рисунок 2.6 – Матриця плутанини

--- Звіт Класифікації (на тестувальних даних) ---

	precision	recall	f1-score	support
Network issues	0.99	0.96	0.97	70
Software Management	0.92	0.94	0.93	70
Account Management	0.99	1.00	0.99	70
Password issues	1.00	1.00	1.00	8
Security incidents	1.00	1.00	1.00	22
File & Storage	1.00	1.00	1.00	24
Application Mgmt	0.94	0.92	0.93	36
accuracy			0.97	300
macro avg	0.98	0.97	0.97	300
weighted avg	0.97	0.97	0.97	300

Рисунок 2.7 – Звіт класифікації

Отримані результати свідчать про високу ефективність розробленого мультикласового класифікатора. Значення F1 Macro 0.966 показує, що модель здатна точно визначати тип ІТ-завдання для переважної більшості запитів, включно з класами, що мають невелику кількість прикладів у тестовому наборі (напр., Password Issues). Аналіз звіту класифікації та матриці плутанини показує, що найскладнішим для моделі виявився клас Application Management, де спостерігалася плутанина з класом Software management. Це може бути пов'язано з семантичною близькістю формулювань. Однак загальна

ефективність моделі є достатньо високою для її використання в рамках ієрархічної системи класифікації та подальшого автоматичного розподілу завдань.

2.6 Розробка алгоритму автоматичного розподілу завдань

Після успішної класифікації ІТ-заявки за допомогою ієрархічної моделі (описаної в підрозділах 2.2-2.5), наступним кроком є її автоматичне призначення відповідному виконавцю. Для цього було розроблено спеціалізований алгоритм розподілу, метою якого є не лише автоматизація процесу, але й оптимізація навантаження на співробітників служби підтримки з урахуванням їхньої компетенції та поточної зайнятості.

Алгоритм приймає на вхід об'єкт завдання, для якого вже визначено його тип (одна з 7 категорій) за допомогою мультикласового класифікатора. Також враховується пріоритетність, асоційована з даним типом завдання. Основними принципами роботи алгоритму є компетентність, навантаження та ескалація. Завдання має бути призначене співробітнику, який володіє необхідними навичками та знаннями для вирішення проблем даного типу. Компетенції працівників визначаються менеджером працівників. Працівник може мати декілька категорій завдань, який він може виконувати. Алгоритм прагне розподіляти завдання таким чином, щоб уникнути надмірного навантаження на окремих виконавців, особливо для завдань з низьким пріоритетом. Рівень навантаження визначається менеджером відділу ІТ підтримки та встановлюється індивідуально для кожного працівника. У випадку, якщо підходящого виконавця не знайдено, передбачено механізм ескалації завдання на рівень менеджера для ручного втручання.

Алгоритм вибору виконавця складається з кількох послідовних кроків. Першим кроком є формування списку всіх потенційно придатних виконавців. Для цього система здійснює запит до бази даних, відбираючи активних

співробітників, які мають роль "Працівник" (Worker) та підтверджену компетенцію саме у тому типі завдань, до якого належить поточна заявка. Також реалізовано можливість виключення певного співробітника зі списку кандидатів (наприклад, у випадку перепризначення завдання).

Далі для кожного співробітника з початкового пулу розраховується інтегральний показник (score), що відображає його придатність для виконання даного конкретного завдання. Цей показник формується на основі двох ключових параметрів – рівень експертизи та рівень поточного навантаження. Рівень експертизи - Числовий показник, що відображає глибину знань та досвід співробітника у вирішенні завдань даного типу (наприклад, від 1 до N). Чим вищий рівень, тим більший внесок у загальний бал. Поточне навантаження – кількість активних завдань, що на даний момент знаходяться у роботі у співробітника. Цей параметр віднімається від загального балу, щоб пріоритетизувати менш завантажених виконавців.

Для запобігання перевантаженню співробітників, алгоритм перевіряє поточне навантаження кожного кандидата відносно встановленого для нього максимального ліміту. Якщо співробітник досяг або перевищив максимальне навантаження, він виключається зі списку кандидатів для завдань з низьким пріоритетом. Однак, для високопріоритетних завдань (наприклад, критичні інциденти) ця перевірка може ігноруватися, щоб забезпечити якнайшвидше призначення завдання кваліфікованому спеціалісту, навіть якщо він тимчасово перевантажений. Передбачено також можливість повністю ігнорувати обмеження максимального навантаження за потреби (наприклад, для ручного призначення менеджером).

Після отримання інтегрального показника score кожного працівника, Серед кандидатів, що залишилися після фільтрації, алгоритм визначає тих, хто має найвищий розрахований показник (score). Якщо таких кандидатів декілька

(з однаково високим балом), вибір між ними здійснюється випадковим чином для забезпечення більш рівномірного розподілу. Обраному співробітнику призначається завдання.

У випадку, якщо після всіх етапів фільтрації та оцінки не вдалося знайти жодного придатного кандидата (наприклад, всі компетентні співробітники перевантажені або неактивні), алгоритм автоматично ініціює процес ескалації. Система ідентифікує активного співробітника з роллю "Менеджер" (Manager) у відповідному ІТ-відділі та призначає завдання йому. Це гарантує, що жодна заявка не залишиться без уваги, і менеджер зможе вручну знайти виконавця або вжити інших заходів. Алгоритм також передбачає обробку виняткових ситуацій, коли не вдається знайти ані виконавця, ані активного менеджера для ескалації, та логує такі випадки для подальшого аналізу.

Таким чином, розроблений алгоритм дозволяє автоматизувати процес призначення ІТ-заявок, враховуючи як компетенції виконавців, так і їх поточне навантаження, а також забезпечує надійний механізм ескалації для нетипових ситуацій, що сприяє більш ефективному та збалансованому розподілу роботи в службі підтримки.

Висновки до розділу

У даному розділі було детально розглянуто та реалізовано ключові компоненти програмного засобу для автоматизації розподілу навантаження служби ІТ-підтримки. Було проаналізовано етапи підготовки даних, включаючи очищення, нормалізацію та формування спеціалізованих наборів для навчання моделей класифікації українськомовних ІТ-заявок.

Було розроблено ієрархічну модель класифікації заявок, що складається з двох рівнів – бінарного та мультикласового класифікатора. Бінарний класифікатор був донавчений для фільтрації нерелевантних запитів. Оцінка ефективності показала високу надійність ($F1 \text{ Macro} \approx 0.96$), що дозволяє

ефективно відсіювати нерелевантні звернення. Мультикласовий класифікатор донавчений для визначення одного з 7 типів ІТ-запитів. Оцінка на незалежному тестовому наборі продемонструвала високу узагальнюючу здатність моделі, незважаючи на незбалансованість тестового набору.

Було наведено опис розробленого алгоритму автоматичного розподілу завдань. Описано його характеристики: врахування типу завдання (визначеного класифікатором), компетенції співробітників, їх поточного навантаження та пріоритету заявки. Також розглянуто механізм ескалації на менеджера у випадку відсутності підходящих виконавців.

Було проаналізовано основні переваги запропонованого ієрархічного підходу до класифікації (надійність обробки OOD-запитів) та алгоритму розподілу (врахування компетенцій та навантаження). Визначено потенційні недоліки або аспекти для подальшого вдосконалення, такі як залежність якості виконання від якості даних про компетенції співробітників.

Таким чином, у другому розділі було закладено теоретичні та практичні основи для реалізації програмного засобу, розроблено ключові інтелектуальні компоненти системи та доведено їхню ефективність на підготовлених даних.

3 РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Опис вимог до програмного забезпечення

На основі аналізу предметної області та постановки задачі було сформульовано набір функціональних та нефункціональних вимог до розроблюваного програмного засобу розподілу навантаження порталу служби підтримки.

3.1.1 Опис функціональним вимог

Загальні вимоги:

- забезпечення безпечного входу користувачів за логіном та паролем;
- система повинна розрізняти ролі (Користувач, Працівник, Менеджер, Адміністратор) та надавати доступ до функціоналу лише відповідно до призначеної ролі.

Вимоги до Користувачів (Users):

- можливість створення нових текстових заявок до служби ІТ-підтримки через веб-інтерфейс;
- доступ до списку власних заявок з відображенням їх поточного статусу та призначеного виконавця;
- можливість скасувати власну активну заявку;
- можливість переглядати та редагувати власні контактні дані (телефон, email) та змінювати пароль;
- можливість переглядати список пристроїв, закріплених за користувачем;
- можливість ініціювати процес повернення пристрою;
- можливість створювати запит на ремонт пристрою з описом проблеми;

- можливість створювати запит на отримання нового особистого пристрою з вказанням типу та характеристик.

Вимоги для Працівників (Worker):

- доступ до персональної панелі, де відображаються всі завдання, призначені даному працівнику, з можливістю сортування та фільтрації;
- можливість переглядати детальний опис завдання, змінювати його статус, додавати коментарі щодо виконання;
- спеціалізовані інтерфейси для обробки завдань типу "Device Management" (з рекомендаціями доступних пристроїв та призначенням їх користувачу/відділу), "Device Return" (з можливістю вказати стан повернутого пристрою та ініціювати ремонт) та "Device Repair" (з можливістю вказати результат ремонту);
- можливість повернути завдання менеджеру для перевірки/перепризначення, вказавши причину;
- можливість відмовитись від виконання завдання з автоматичним пошуком заміни та вказанням причини;
- можливість редагувати власний профіль.

Вимоги до Менеджерів (Manager):

- доступ до панелі, що відображає всі завдання, призначені підлеглим працівникам, з можливістю фільтрації та перегляду деталей;
- можливість вручну змінити тип завдання (якщо автоматична класифікація була неточною) та/або перепризначити завдання іншому виконавцю зі своєї команди;
- перегляд списку підлеглих працівників, редагування їх профілів, включаючи зміну статусу, дати недоступності та управління їхніми компетенціями;
- перегляд списку пристроїв, закріплених за відділом та підлеглими;

- можливість ініціювати повернення пристроїв відділу;
- можливість створювати запити на нові пристрої для потреб відділу з вказанням характеристик;

- можливість редагувати власний профіль.

Вимоги до Адміністраторів (Admin):

- повний CRUD (Create, Read, Update, Deactivate) функціонал для облікових записів всіх користувачів системи (включаючи призначення ролей, відділів, менеджерів);
- можливість керувати довідниками відділів, ролей, типів завдань, типів пристроїв та їх атрибутів.

Вимоги до автоматизації (Системні):

- при створенні нової заявки система повинна автоматично визначати її тип за допомогою інтегрованої ієрархічної моделі NLP;
- на основі визначеного типу завдання, система повинна автоматично призначати виконавця за розробленим алгоритмом, враховуючи компетенції, навантаження та пріоритет.

3.1.2 Опис нефункціональних вимог

- час відповіді веб-інтерфейсу на типові дії користувача не повинен перевищувати 2 секунди. Процес автоматичної класифікації та призначення завдання має виконуватися за час, що є непомітним для користувача (менше 1 секунди);
- забезпечення захисту від несанкціонованого доступу, хешування паролів, захист від поширених веб-вразливостей;
- інтерфейс має бути інтуїтивно зрозумілим для користувачів з різним рівнем технічної підготовки;
- код системи повинен бути добре структурованим, документованим та допускати легке внесення змін чи додавання нового функціоналу.

3.2 Вибір технологій

Для реалізації програмного засобу розподілу навантаження порталу служби підтримки було обрано стек технологій, що відповідає сформульованим функціональним та нефункціональним вимогам, а також забезпечує ефективну інтеграцію з розробленими моделями машинного навчання для класифікації тексту.

Як основну мову розробки обрано Python завдяки її багатій екосистемі бібліотек для веб-розробки (Flask), роботи з даними (Pandas), взаємодії з базами даних (SQLAlchemy) та, що особливо важливо для даного проєкту, машинного навчання (Transformers, PyTorch/TensorFlow).

Веб-фреймворк Flask використано як основу для побудови веб-додатку. Flask є мікрофреймворком, що надає гнучкість у структуруванні проєкту та виборі додаткових компонентів. Його простота та мінімалістичність дозволили створити додаток, чітко розділений на логічні модулі, що відповідає вимогам до підтримуваності та розширюваності.

PostgreSQL – реляційна СУБД обрана для надійного зберігання структурованих даних системи: інформації про користувачів (працівників), їх ролі та відділи, типи завдань та їх компетенції, самі заявки (завдання), призначені пристрої та історію змін статусів.

Бібліотека SQLAlchemy використовується для взаємодії Python-коду з реляційною базою даних. SQLAlchemy дозволяє визначати структуру бази даних за допомогою Python-класів та виконувати запити до БД, оперуючи об'єктами, що значно спрощує розробку та підвищує надійність роботи з даними. У коді проєкту SQLAlchemy активно використовується для вибірки, фільтрації, оновлення та створення записів.

Бібліотека Transformers (Hugging Face) – це ключова бібліотека для інтеграції NLP-функціоналу. Вона використовується для завантаження перед-

навчених трансформерних, їх донавчання (fine-tuning) на специфічних даних ІТ-заявок та подальшого використання для автоматичної класифікації тексту.

В інтерфейсі користувача було використано стандартні веб-технології: HTML для структури сторінок, CSS для стилізації та JavaScript для реалізації динамічної поведінки на стороні клієнта (наприклад, для асинхронного завантаження списку менеджерів при створенні/редагуванні користувача). Шаблонізатор Jinja2, інтегрований у Flask, використовується для генерації HTML-сторінок на основі даних, переданих з серверної частини.

Для аутентифікації та управління сесіями було використано Flask-Login. Це розширення Flask забезпечує реалізацію механізмів входу користувачів, перевірку їх автентичності при кожному запиті та управління сесіями. У поєднанні з власними декораторами забезпечується також контроль доступу на основі ролей.

3.3 Розробка архітектури

Програмний засіб розроблено як монолітний вебдодаток на базі фреймворку Flask. Архітектура системи значною мірою відповідає шаблону Model-View-Controller (MVC), адаптованому під специфіку Flask, який часто описують як Model-Template-View (MTV) або просто використовують MVC термінологію з певними уточненнями [7]. Основні компоненти системи та їх взаємодія показані на рисунку 3.1.

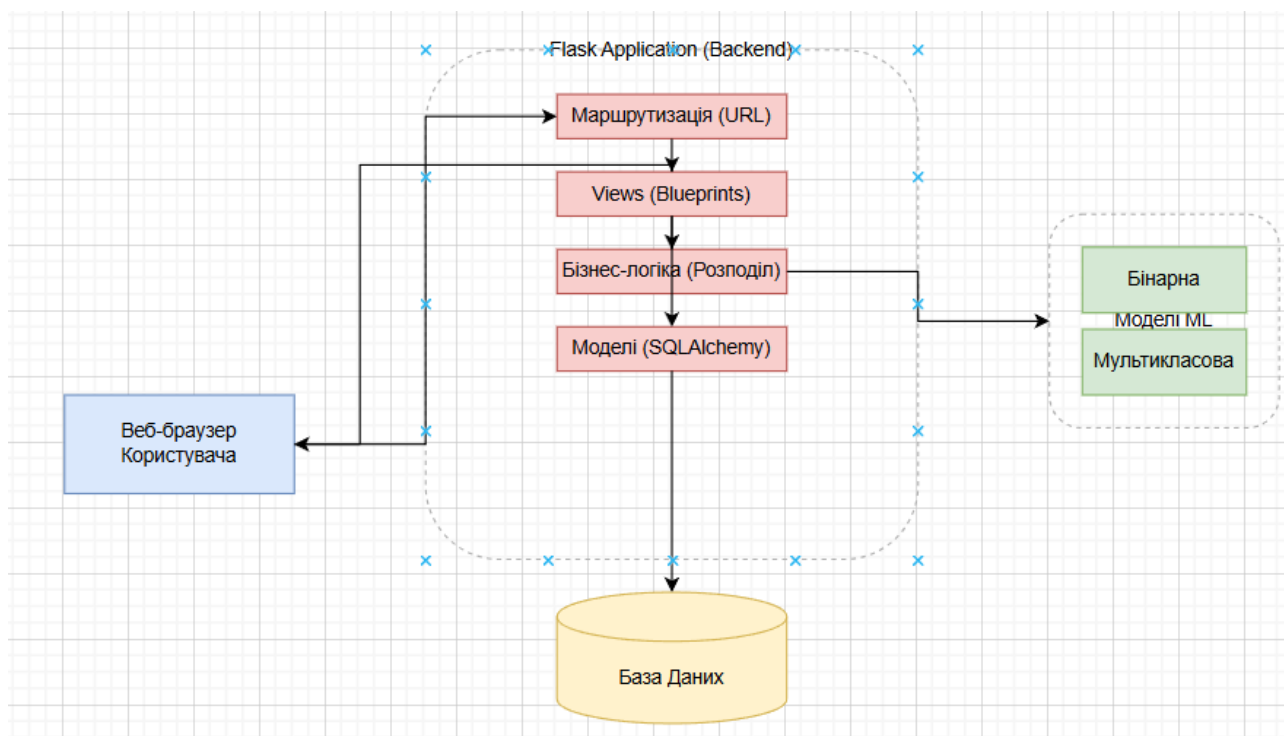


Рисунок 3.1 – Основні компоненти системи та їх взаємодія

Ключовими компонентами архітектури є Модель (Model), Представлення (View/Template) та Контролер (Controller/View Function).

Шар Моделі відповідає за дані та бізнес-логіку, пов'язану безпосередньо з даними. Він представлений класами SQLAlchemy, які описують структуру таблиць у реляційній базі даних (користувачі, ролі, відділи, завдання, пристрої, їхні статуси, історичні записи тощо). Ці класи інкапсулюють взаємодію з базою даних, надаючи об'єктно-орієнтований інтерфейс для CRUD операцій. Також цей шар включає логіку роботи з даними, таку як зв'язки між моделями, методи для отримання специфічних даних та логіку обробки історії змін.

Шар Представлення (View) відповідає за відображення інформації користувачеві та отримання від нього введення. Він складається з HTML-шаблонів (Jinja2), які є динамічними файлами, що визначають структуру веб-сторінок і заповнюються даними з Контролера, та статичних файлів (CSS,

JavaScript), що забезпечують візуальне оформлення та інтерактивність інтерфейсу на стороні клієнта.

Шар Контролера у Flask реалізовано функціями-обробниками маршрутів у файлах `*_views.py`. Ці функції виконують маршрутизацію HTTP-запитів за певними URL-адресами, обробляють дані з форм або URL-параметрів, звертаються до SQLAlchemy-моделей для роботи з базою даних, запускають специфічні функції бізнес-логіки (наприклад, алгоритм розподілу завдань `choose_worker`), викликають функції для класифікації тексту за допомогою моделей ML. формують відповідь для Представлення (рендерінг HTML або JSON) та контролюють доступ за допомогою декораторів (`@login_required`, `@role_required`).

Для кращої організації коду та розділення функціоналу за ролями користувачів використано механізм Flask Blueprints. Кожен файл визначає окремий Blueprint, групуючи маршрути та логіку для конкретної ролі, що підвищує читабельність та підтримуваність коду.

Компоненти Машинного Навчання представлені двома попередньо навченими та збереженими моделями `transformers`, які завантажуються додатком і використовуються Контролером для класифікації тексту. База Даних PostgreSQL слугує зовнішнім сховищем для всієї персистентної інформації додатку.

Взаємодія компонентів відбувається за типовим потоком: користувач взаємодіє з Представленням (веб-сторінкою) та відправляє запит на певний URL. Flask маршрутизує запит до відповідної функції Контролера. Контролер обробляє дані, взаємодіє з Моделлю для роботи з базою даних, за потреби викликає Моделі ML або іншу бізнес-логіку, а потім передає результат назад Представленню для відображення користувачеві.

Обрана архітектура забезпечує чіткий поділ відповідальності, що відповідає нефункціональним вимогам до підтримуваності та розширюваності системи. Використання ORM спрощує роботу з базою даних, а Flask Blueprints сприяють модульності кодової бази [8]. Інтеграція моделей машинного навчання реалізована на рівні Контролера/бізнес-логіки.

3.4 Інтерфейс користувача

Вебзастосунок передбачає інтерфейси для чотирьох ролей користувачів: адміністратора, менеджера, замовника та виконавця. Користування вебзастосунком починається з сторінки авторизації (рисунок 3.2). Далі після отримання ролі працівника відкриваються відповідні початкові сторінки кожної ролі.

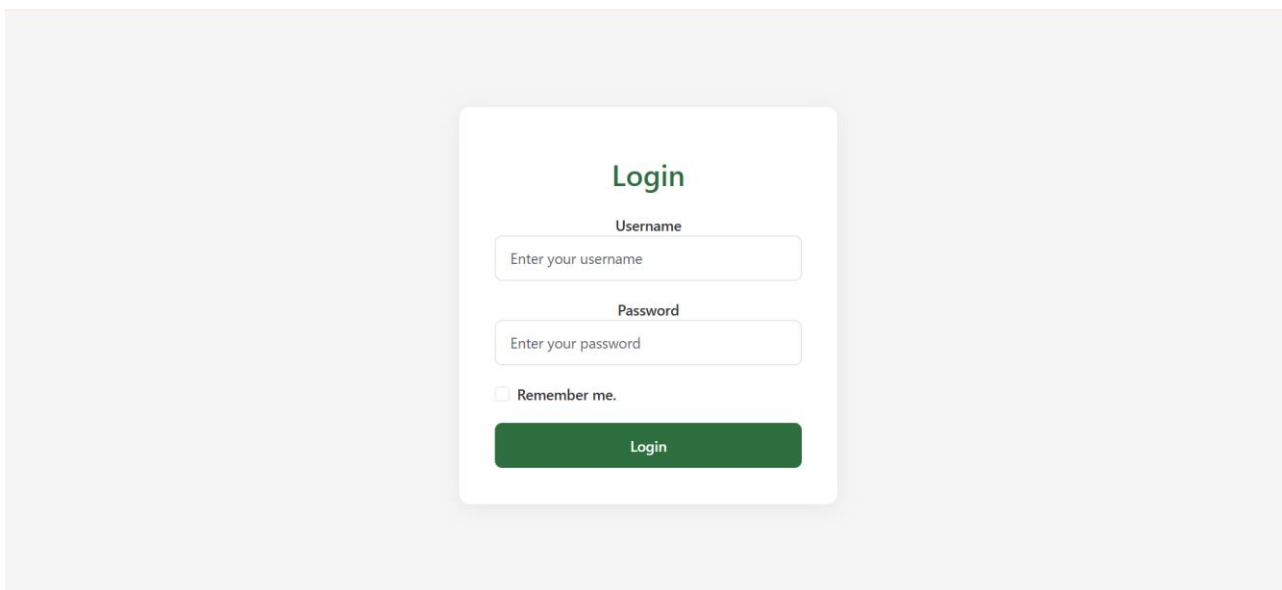
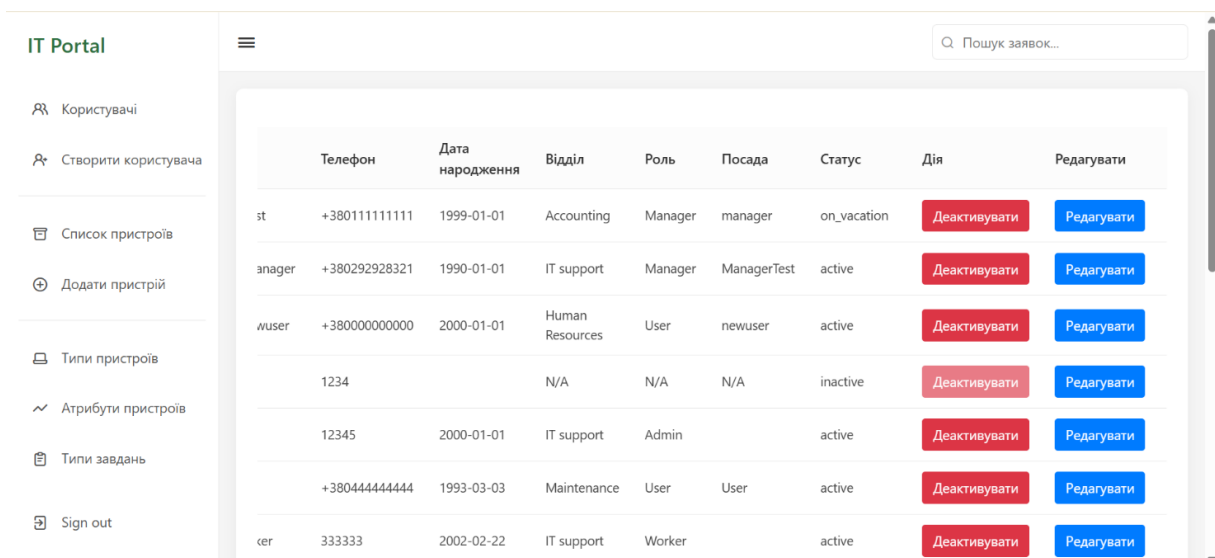


Рисунок 3.2 – Сторінка авторизації

Інтерфейс адміністратора складається з:

- головної сторінки, що містить список всіх користувачів системи (рисунок 3.3);
- сторінок редагування користувачів (рисунок 3.4);
- сторінки створення нового користувача (рисунок 3.5);

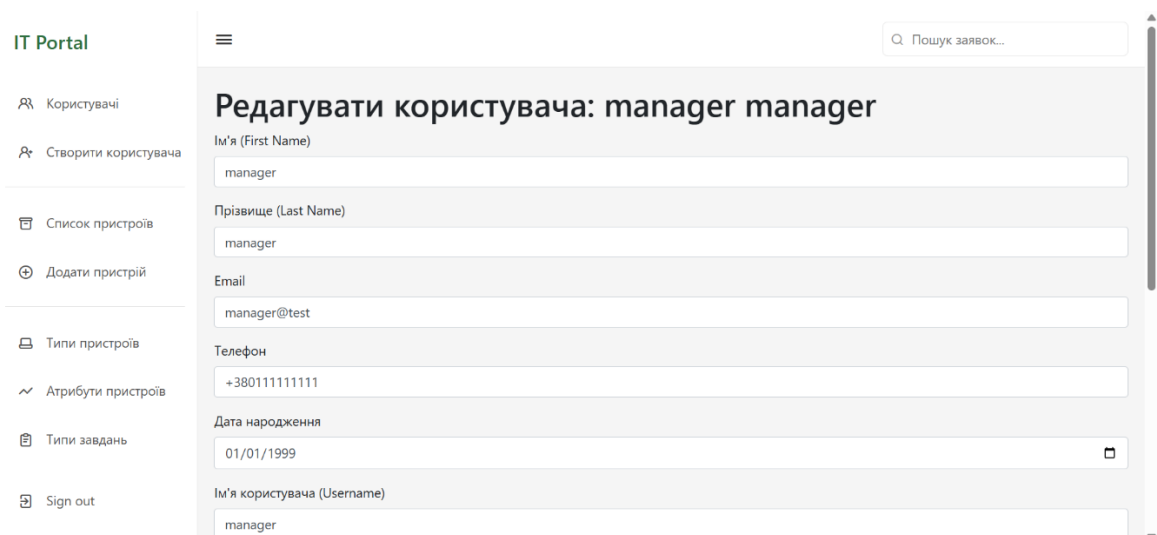
- сторінки, що містить список всіх пристроїв системи та дані про них (рисунок 3.6);
- сторінки для редагування інформації про пристрій (рисунок 3.7);
- сторінку для додавання нового пристрою до системи (рисунок 3.8);
- сторінку для створення та перегляду типів пристроїв (рисунок 3.9);
- сторінку для створення та перегляду атрибутів пристроїв (рисунок 3.10);
- сторінку для створення та перегляду типів завдань (рисунок 3.11).



The screenshot shows the 'IT Portal' administrator interface. On the left is a sidebar with navigation links: Користувачі, Створити користувача, Список пристроїв, Додати пристрій, Типи пристроїв, Атрибути пристроїв, Типи завдань, and Sign out. The main content area displays a table of users with columns: Телефон, Дата народження, Відділ, Роль, Посада, Статус, Дія, and Редагувати. The table contains seven rows of user data.

Телефон	Дата народження	Відділ	Роль	Посада	Статус	Дія	Редагувати
st +380111111111	1999-01-01	Accounting	Manager	manager	on_vacation	Деактивувати	Редагувати
anager +380292928321	1990-01-01	IT support	Manager	ManagerTest	active	Деактивувати	Редагувати
wuser +380000000000	2000-01-01	Human Resources	User	newuser	active	Деактивувати	Редагувати
1234		N/A	N/A	N/A	inactive	Деактивувати	Редагувати
12345	2000-01-01	IT support	Admin		active	Деактивувати	Редагувати
+380444444444	1993-03-03	Maintenance	User	User	active	Деактивувати	Редагувати
cer 333333	2002-02-22	IT support	Worker		active	Деактивувати	Редагувати

Рисунок 3.3 – Головна сторінка адміністратора



The screenshot shows the 'IT Portal' administrator interface with the 'Редагувати користувача: manager manager' form. The form contains fields for: Ім'я (First Name) (manager), Прізвище (Last Name) (manager), Email (manager@test), Телефон (+380111111111), Дата народження (01/01/1999), and Ім'я користувача (Username) (manager). The sidebar and search bar are also visible.

Рисунок 3.4 – Сторінка редагування інформації про користувача

IT Portal

- Користувачі
- Створити користувача
- Список пристроїв
- Додати пристрій
- Типи пристроїв
- Атрибути пристроїв
- Типи завдань
- Sign out

127.0.0.1:5000/admin/admin_dashboard

Створення нового користувача

Ім'я (First Name)
Введіть ім'я

Прізвище (Last Name)
Введіть прізвище

Email
Введіть email

Телефон
+380

Дата народження
dd/mm/yyyy

Ім'я користувача (Username)
Введіть username

Рисунок 3.5 – Сторінка створення нового користувача

IT Portal

- Користувачі
- Створити користувача
- Список пристроїв
- Додати пристрій
- Типи пристроїв
- Атрибути пристроїв
- Типи завдань
- Sign out

127.0.0.1:5000/admin/admin_dashboard

Список всіх пристроїв

ID	Назва	Тип	Серійний номер	Статус	Поточний власник	Дії
51	HP ProBook 450 G8	Ноутбук	SN-LAP-001	available	На складі	Редагувати Списати
52	Dell Latitude 5520	Ноутбук	SN-LAP-002	available	На складі	Редагувати Списати
53	Lenovo ThinkPad E15	Ноутбук	SN-LAP-003	available	На складі	Редагувати Списати
54	Apple MacBook Pro 14	Ноутбук	SN-LAP-004	decommissioned	User User	Редагувати Списати

Рисунок 3.6 – Сторінка, що містить список всіх пристроїв системи

IT Portal

Пошук заявок...

Додавання нового пристрою

Назва пристрою (Модель)
Наприклад, HP ProBook 450 G8

Серійний номер
Унікальний серійний номер

Тип пристрою
Оберіть тип пристрою...

Характеристики
Будь ласка, спочатку оберіть тип пристрою.

Зберегти пристрій Скасувати

127.0.0.1:5000/admin/create_user

Рисунок 3.7 – Сторінка для додавання нового пристрою до системи

IT Portal

Пошук заявок...

Редагувати пристрій: HP ProBook 450 G8

Назва пристрою (Модель)
HP ProBook 450 G8

Серійний номер
SN-LAP-001

Тип пристрою
Ноутбук

Характеристики
Модель
Процесор

Рисунок 3.8 – Сторінка для редагування інформації про пристрій

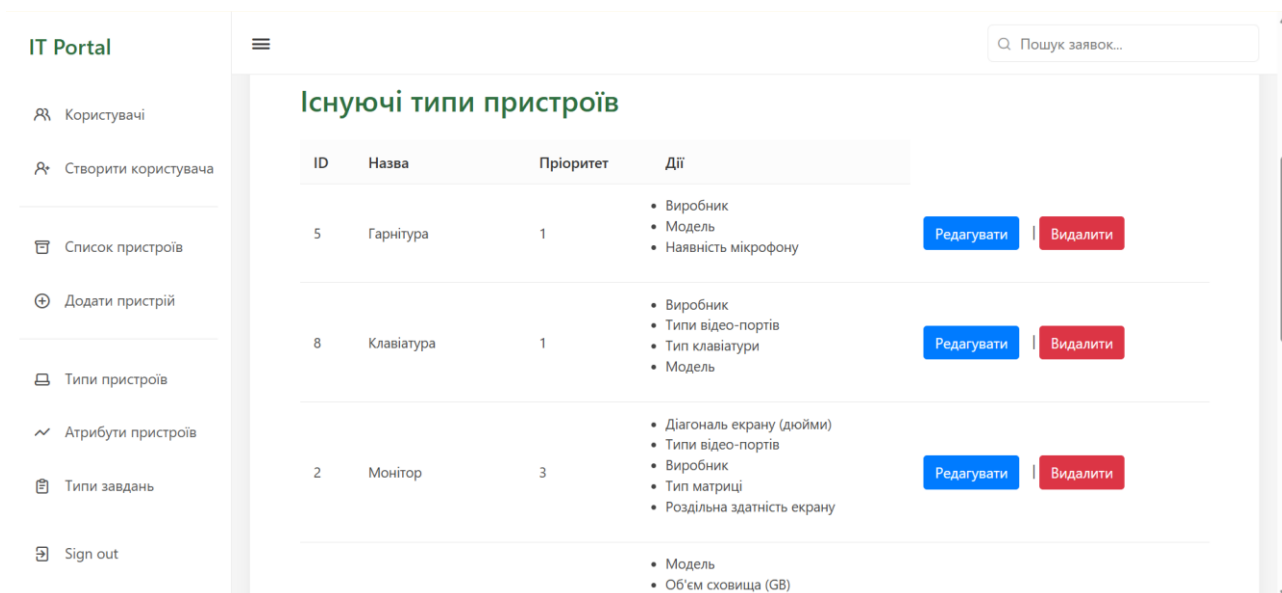


Рисунок 3.9 – Сторінка для створення та перегляду типів пристроїв

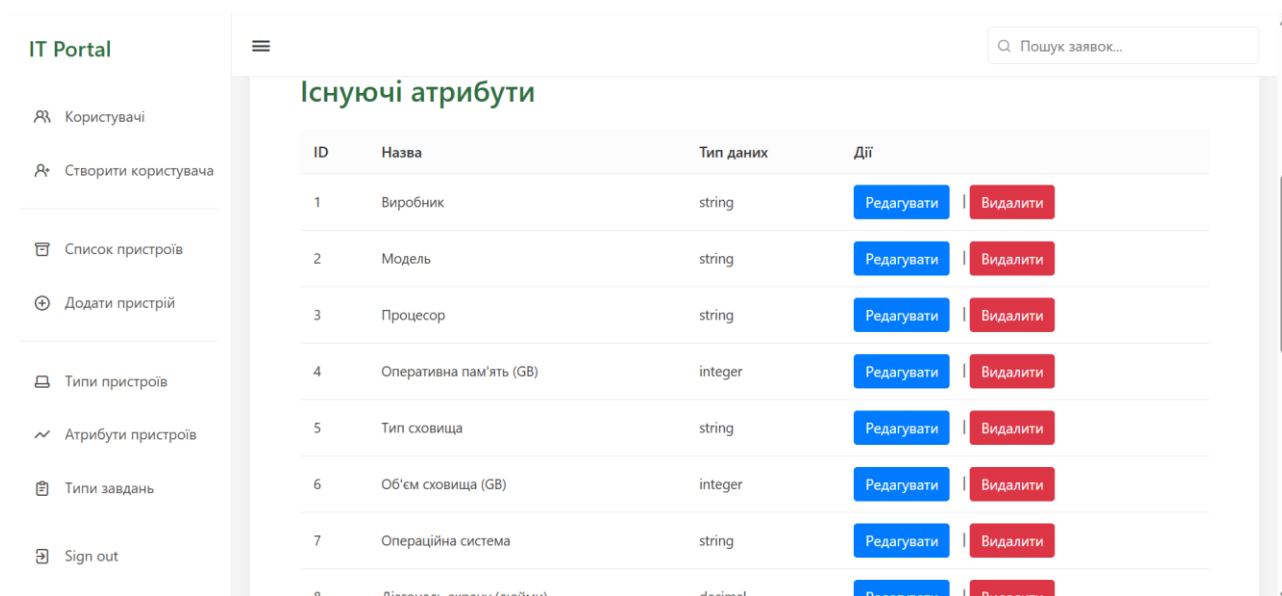


Рисунок 3.10 – Сторінка для створення та перегляду атрибутів пристроїв

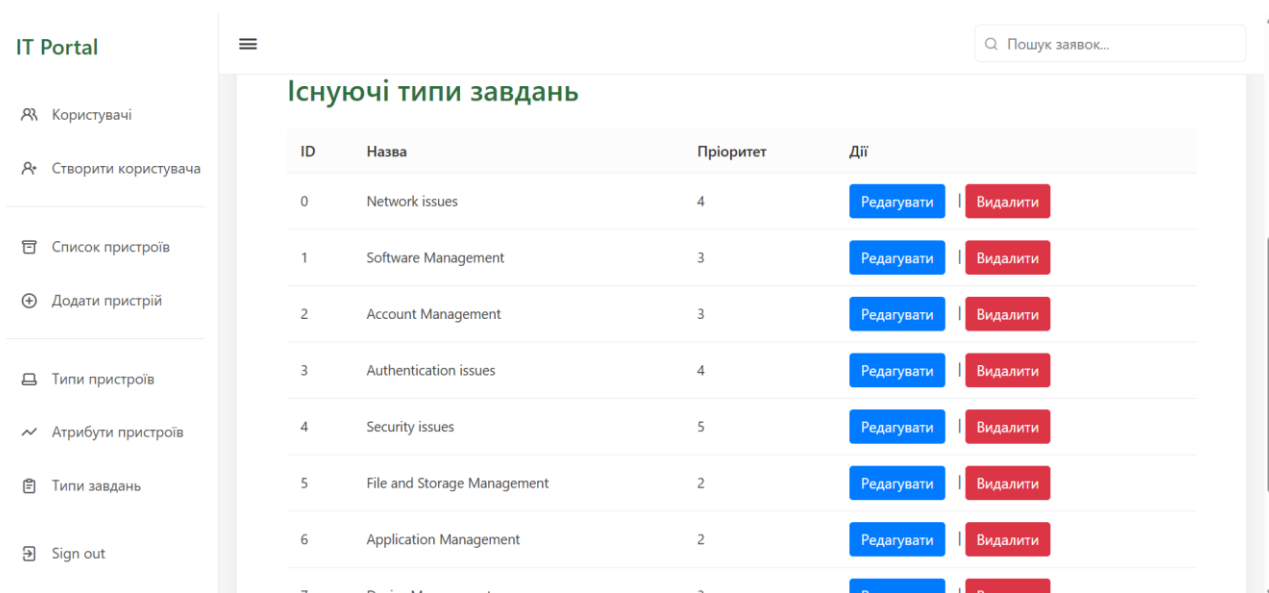


Рисунок 3.11 – Сторінка для створення та перегляду типів завдань

Інтерфейс менеджера складається з:

- головної сторінки, що містить всі заявки, направлені до відділу, з їх пріоритетами (рисунок 3.12);
- сторінок перегляду заявок з можливістю визначення нового типу та/або виконавця (рисунок 3.13);
- сторінки з заявками, що були ескаловані до менеджера (рисунок 3.14);
- сторінки для створення заявок та перегляду пристроїв відділу (рисунок 3.15);
- сторінки перегляду працівників відділу (рисунок 3.16);
- сторінки для перегляду та редагування компетентностей працівників (рисунок 3.17);
- особистої сторінки менеджера з можливістю редагування особистої інформації (рисунок 3.18).

IT Portal

Керування заявками

Заявки без виконавця

Пристрої відділу

Мої працівники

Мій профіль

Вийти

Пошук завдань...

ID	Пріоритет	Тип	Замовник	Відділ замовника	Виконавець	Статус завдання	Метод призначення	Дата призначення	Дії
37	Високий (5)	Security issues	User User	Maintenance	Worker11 Worker11	Active	Automatic	30.10.2025 10:59	Переглянути
40	Високий (5)	Security issues	User User	Maintenance	Worker11 Worker11	Active	Automatic	30.10.2025 10:59	Переглянути
38	Високий (4)	Authentication issues	User User	Maintenance	Worker6 Worker6	Active	Automatic	30.10.2025 10:59	Переглянути
45	Високий (4)	Authentication issues	User User	Maintenance	Worker2 Worker2	Active	Manual	26.11.2025 12:11	Переглянути
46	Високий (4)	Network issues	User User	Maintenance	Worker8 Worker8	Active	Automatic	26.11.2025 14:29	Переглянути
47	Високий (4)	Device Repair	ManagerTest ManagerTest	IT support	Worker11 Worker11	Active	Automatic	30.11.2025 12:23	Переглянути
15	Середній (3)	Device Management	ManagerTest ManagerTest	IT support	Worker4 Worker4	Active	Automatic	17.10.2025 19:48	Переглянути

Рисунок 3.12 – Головна сторінка менеджера

IT Portal

- Керування заявками
- Заявки без виконавця
- Пристрої відділу
- Мої працівники
- Мій профіль
- Вийти

Редагування заявки

Текст заявки (оригінал):

Система змушує мене змінити пароль, але не приймає жоден з моїх нових варіантів, каже 'пароль не відповідає вимогам'.

Тип завдання:

Security issues

Виконавець:

Worker11 Worker11

Зберегти зміни

До списку завдань

Рисунок 3.13 – Сторінка заявки

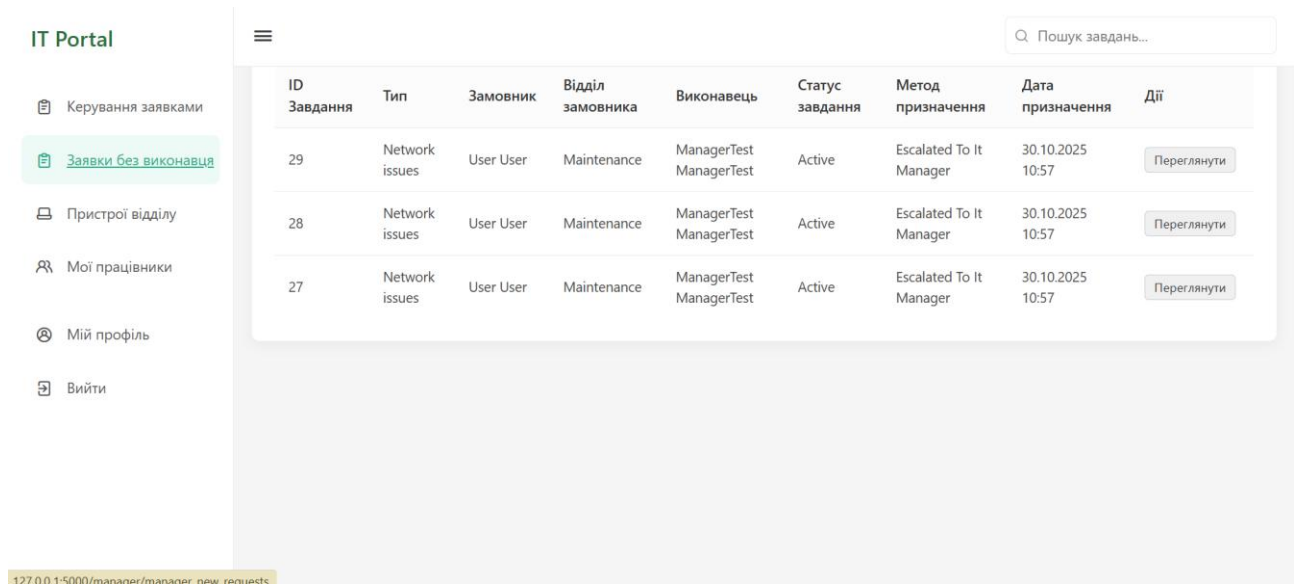


Рисунок 3.14 – Сторінка з заявками без виконавця

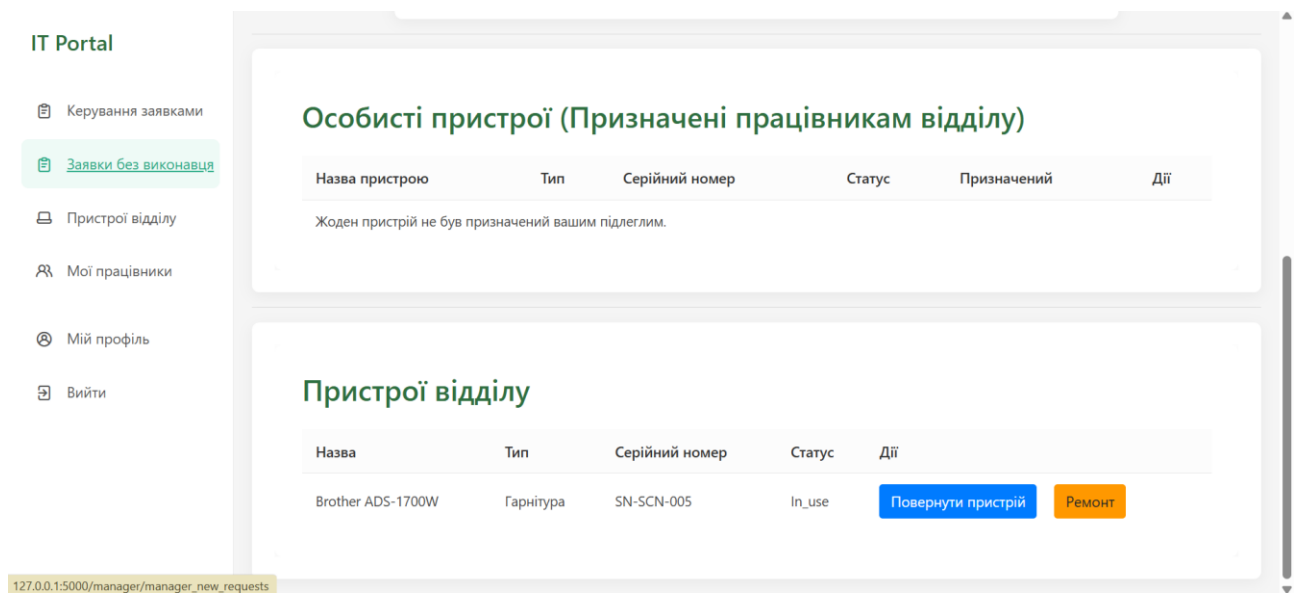


Рисунок 3.15 – Сторінка для створення заявки на отримання нового пристрою

IT Portal

Керування заявками
Заявки без виконавця
Пристрої відділу
Мої працівники
Мій профіль
Вийти

Пошук працівника...

ID	Прізвище	Ім'я	Email	Телефон	Посада	Статус	Завд.	Компетенції	Дії
10	Worker	Worker	worker@worker	333333		Active	1	File and Storage Management Network issues Security issues Software Management Device Management	Редагувати
20	Worker10	Worker10	worker10@worker	+380810101010	Worker10	Active	2	Application Management Software Management Account Management	Редагувати
21	Worker11	Worker11	worker11@worker	+380112112211	Worker11	Active	4	Security issues Device Repair	Редагувати
22	Worker12	Worker12	worker12@worker	+380012121212	Worker12	Active	2	Software Management Security issues Device Repair	Редагувати

Authentication

Рисунок 3.16 – Сторінка для перегляду працівників відділу

IT Portal

Керування заявками
Заявки без виконавця
Пристрої відділу
Мої працівники
Мій профіль
Вийти

Налаштування навантаження

Максимальна кількість активних завдань:

20

Якщо залишити пустим, ліміту не буде.

Компетенції та Експертиза

Оберіть типи завдань та вкажіть рівень експертизи (1 - Новачок, 5 - Експерт).

☐ Account Management	Рівень: 1
☐ Application Management	Рівень: 1
☐ Authentication issues	Рівень: 1
☒ Device Management	Рівень: 2

Рисунок 3.17 – Сторінка для перегляду та редагування компетентностей працівника

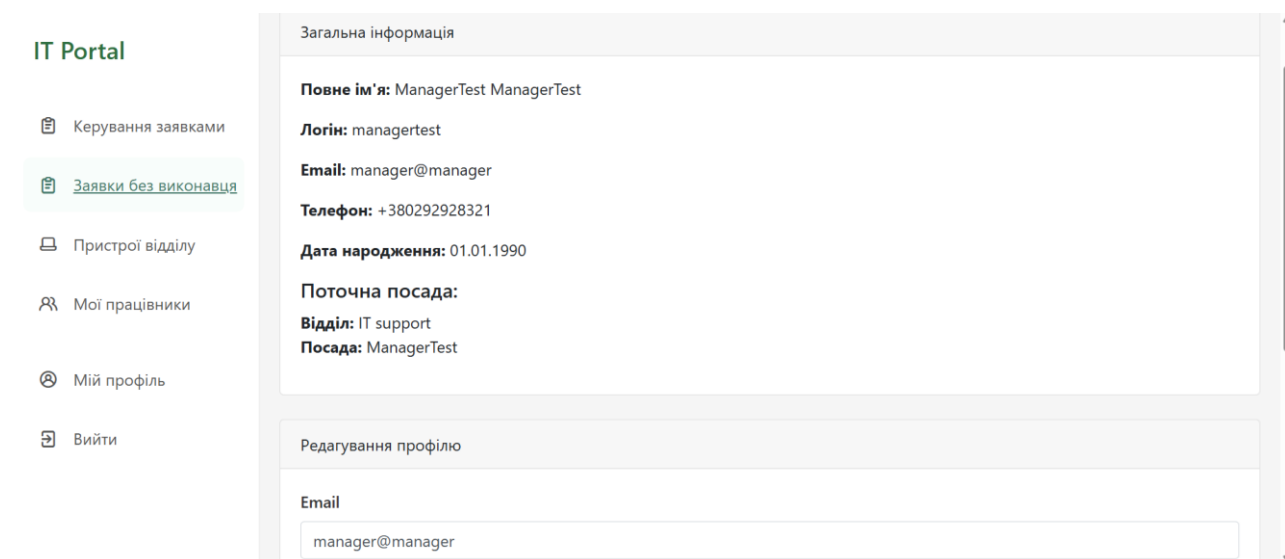


Рисунок 3.18 – Особиста сторінка менеджера

Інтерфейс користувача складається з:

- головної сторінки, що містить всі заявки, створені користувачем (рисунок 3.18);
- сторінки для створення нової заявки (рисунок 3.19);
- сторінки, що містить всі пристрої, надані користувачу (рисунок 3.20);
- сторінки для створення нової заявки для отримання пристрою (рисунок 3.21);
- особистої сторінки користувача з можливістю редагування персональної інформації (рисунок 3.22).

IT Portal

- ✓ Мої заявки
- Створити заявку
- Мої пристрої
- Запит на пристрій
- Мій профіль
- Вийти

ID	Тип	Текст	Статус	Створено	Завершено	Виконавець	Дії
46	Network issues	Постійно розривається VPN-з'єднання. Не можу стабільно працювати з дому	New	26.11.2025 14:29	-	Worker8 Worker8	Скасувати
45	Authentication issues	Добрий день. Проблема з мережею у відділі логістики. Не можемо передати...	New	30.10.2025 13:01	-	Worker2 Worker2	Скасувати
44	Application Management	Наша Jira працює неймовірно повільно, особливо при відкритті дашбордів. Це...	Closed	30.10.2025 11:00	-	Worker10 Worker10	
43	Application Management	Microsoft Teams постійно 'вилітає' під час відеодзвінків. Вже...	New	30.10.2025 11:00	-	Worker10 Worker10	Скасувати
42	File and Storage Management	На моєму хмарному сховищі (OneDrive) закінчується місце. Чи можна збільшити ліміт?	New	30.10.2025 11:00	-	Worker Worker	Скасувати
41	File and Storage Management	Не можу отримати доступ до спільного диска (диск G:). Пише 'відмовлено в...'	New	30.10.2025 10:59	-	Worker8 Worker8	Скасувати
40	Security issues	Мій комп'ютер почав працювати дуже повільно, а антивірус показує...	New	30.10.2025 10:59	-	Worker11 Worker11	Скасувати

Рисунок 3.18 – Головна сторінка користувача

IT Portal

- ✓ Мої заявки
- Створити заявку
- Мої пристрої
- Запит на пристрій
- Мій профіль
- Вийти

Створення нової заявки

Будь ласка, детально опишіть вашу проблему або запит. Система автоматично визначить тип завдання та призначить відповідального виконавця.

Опис проблеми:

Наприклад: Не можу підключитися до мережевого принтера у кабінеті 105. Потрібна допомога з налаштуванням.

Відправити заявку

Скасувати

Рисунок 3.19 – Сторінка для створення нової заявки

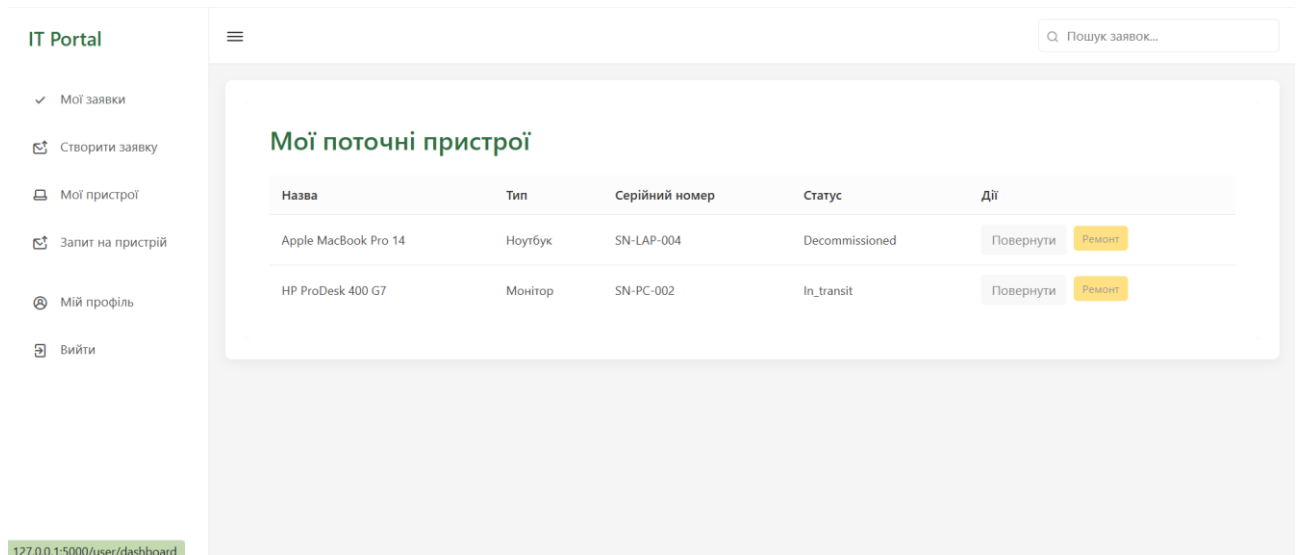


Рисунок 3.20 – Сторінка, що містить перелік всіх пристроїв

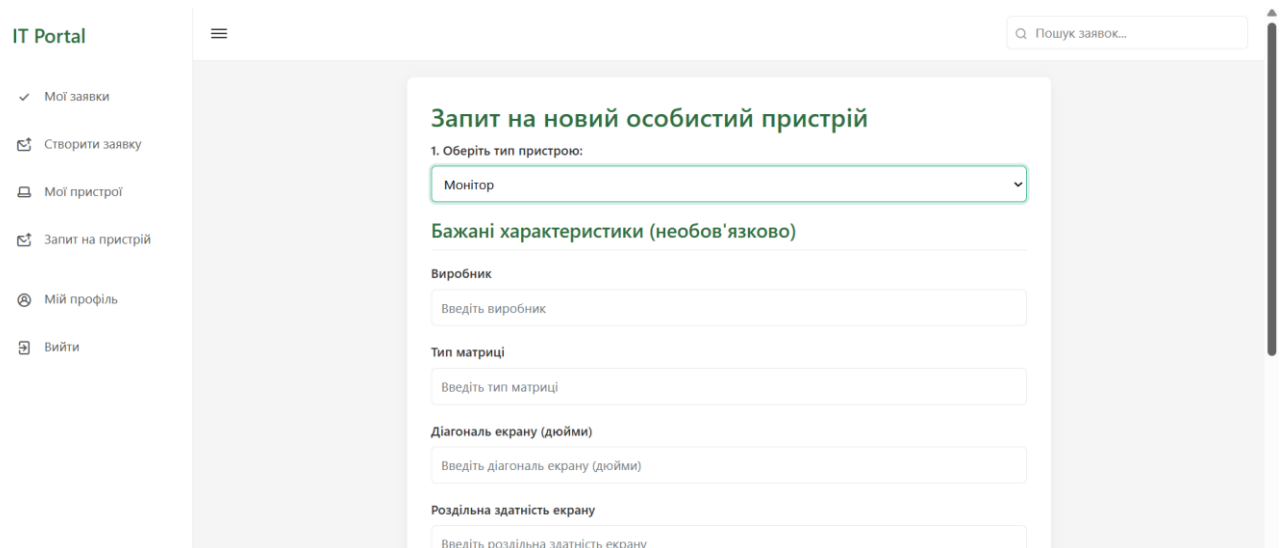


Рисунок 3.21 – Сторінка для запити нового пристрою

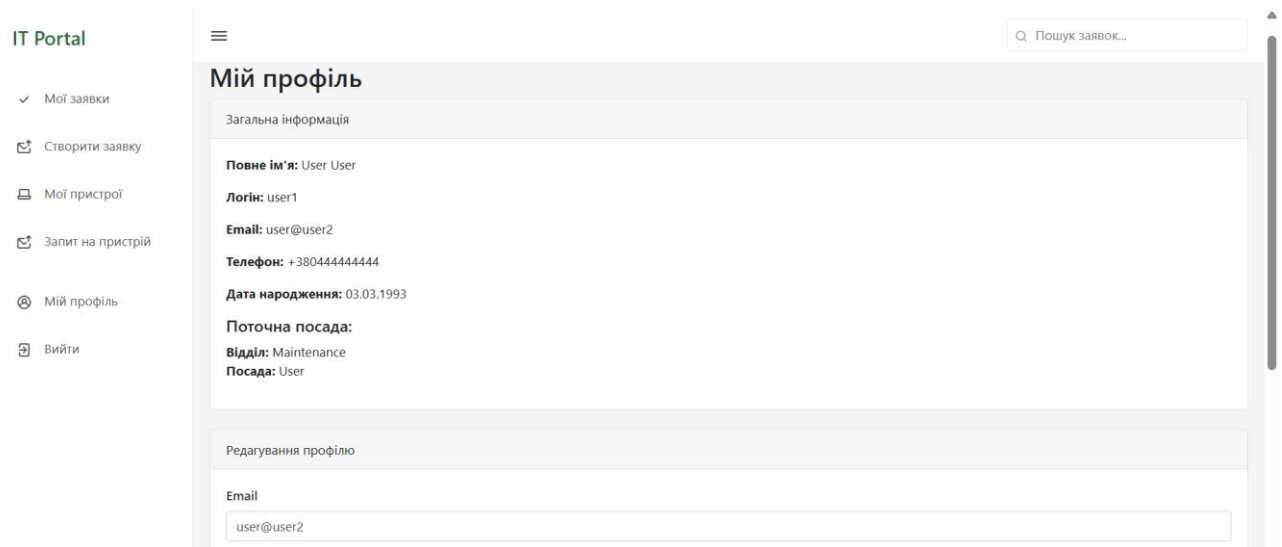


Рисунок 3.22 – Особиста сторінка користувача

Інтерфейс працівника складається з:

- головної сторінки, що містить всі задачі працівника (рисунок 3.23);
- сторінок заявки з можливістю відмови від заявки та повернення заявки до менеджера (рисунок 3.24);
- сторінок заявок на отримання пристроїв, де можна призначити пристрій користувачу (рисунок 3.25);
- особистої сторінки працівника з можливістю редагування персональної інформації (рисунок 3.26).

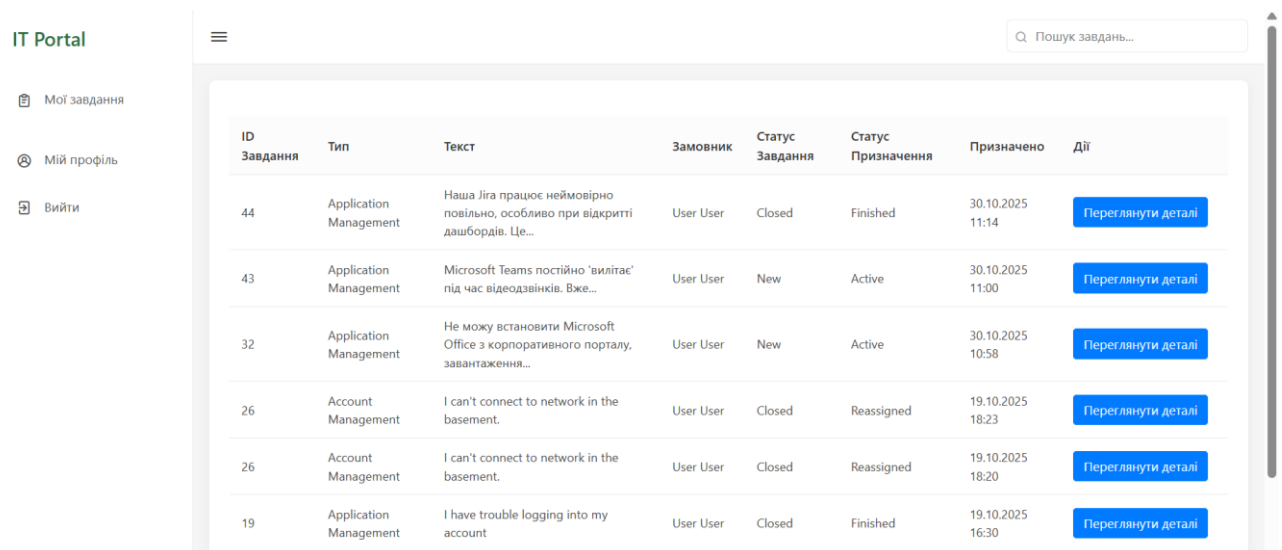


Рисунок 3.23 - Головна сторінка працівника

IT Portal

- Мої завдання
- Мій профіль
- Вийти

Завдання #44: Application Management

Замовник: User User (Відділ: Maintenance)

Створено: 30.10.2025 11:00

Текст заявки:
Наша Jira працює неймовірно повільно, особливо при відкритті дашбордів. Це проблема тільки у мене?

Оновити статус / Додати коментар

Новий статус:

Active

Коментар (необов'язково):

Зберегти зміни

Повернутися до списку

Рисунок 3.24 – Сторінка обробки заявки

IT Portal

- Мої завдання
- Мій профіль
- Вийти

Призначити пристрій

Система проаналізувала запит та відсортувала доступні пристрої за рівнем відповідності.

Рекомендовані пристрої:

-- Оберіть пристрій --

-- Оберіть пристрій --

Epson Perfection V39 (SN: SN-SCN-001) - Відповідність: 0 / 1
Fujitsu ScanSnap iX1400 (SN: SN-SCN-004) - Відповідність: 0 / 1
HP ScanJet Pro 2500 f1 (SN: SN-SCN-003) - Відповідність: 0 / 1

Рисунок 3.25 – Сторінка обробки заявки на отримання пристроїв

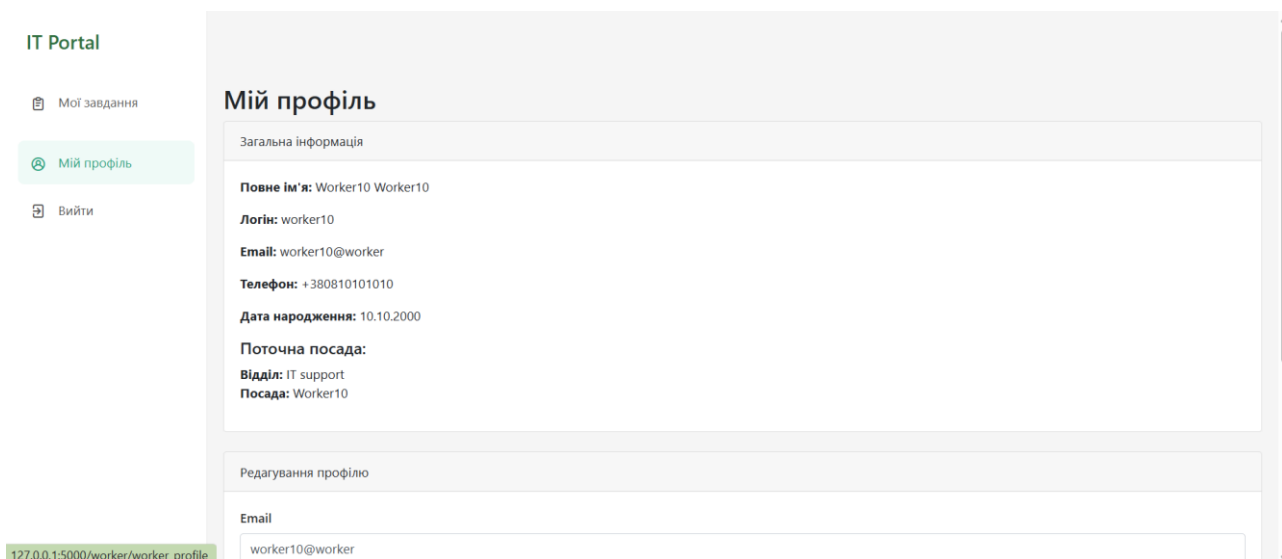


Рисунок 3.26 – Особиста сторінка працівника

3.5 Інструкції по розгортанню ПЗ

Для забезпечення надійності, переносимості та спрощення процесу інсталяції розробленого програмного засобу було обрано стратегію контейнеризації [9]. Система розгортається як набір ізольованих сервісів, що взаємодіють між собою у віртуальній мережі. Цей підхід дозволяє абстрагуватися від особливостей операційної системи хост-машини та гарантує, що середовище виконання буде ідентичним середовищу розробки.

Архітектура розгортання базується на інструменті оркестрації контейнерів Docker Compose і складається з двох основних компонентів. Першим компонентом є сервіс веб-додатку, який побудований на базі легковагого образу операційної системи Linux з попередньо встановленим інтерпретатором Python. Цей контейнер містить вихідний код програми, статичні файли, шаблони інтерфейсу та включає всі необхідні залежності, зокрема бібліотеки для роботи з веб-фреймворком, базою даних та бібліотеки для машинного навчання. Другим компонентом є сервіс бази даних, що використовує офіційний стабільний образ системи управління базами даних PostgreSQL. Він відповідає за зберігання всієї інформації системи, такої як дані

користувачів, заявки та історія розподілу. Дані бази зберігаються у спеціальному томі (Docker Volume), що гарантує їх збереження навіть у випадку перезапуску або повторного створення контейнерів.

Вимоги до середовища для розгортання є мінімальними, оскільки всі залежності інкапсульовані всередині контейнерів. Для успішного запуску на хост-системі, якою може виступати як локальний комп'ютер, так і виділений сервер, необхідна наявність лише середовища виконання контейнерів Docker Engine та інструменту оркестрації Docker Compose. Наявність встановлених Python, PostgreSQL або бібліотек машинного навчання на самій хост-машині не вимагається.

Процедура розгортання системи є значною мірою автоматизованою і розпочинається з підготовки файлової системи. На цільову машину необхідно скопіювати директорію проєкту, що містить вихідний код, конфігураційні файли розгортання (Dockerfile, docker-compose.yml) та файл залежностей. Окрему увагу слід приділити підготовці моделей машинного навчання. Оскільки файли ваг навчених моделей є об'ємними, вони розміщуються у спеціально відведеній директорії, яка згодом монтується всередину контейнера як зовнішній том. Це дозволяє уникнути збільшення розміру основного образу програми та спрощує оновлення моделей.

Наступним етапом є побудова та запуск контейнерів. Ініціалізація системи виконується однією командою `docker-compose up --build -d`, яка запускає процес побудови. Під час цього процесу система автоматично завантажує базові образи ОС, інсталує необхідні системні бібліотеки для компіляції пакетів, а також завантажує та встановлює всі Python-залежності згідно зі специфікацією проєкту. Також створюється ізольована внутрішня мережа для безпечної комунікації між веб-сервером та базою даних. Після виконання цієї команди сервіси стають активними у фоновому режимі.

Для початку повноцінної роботи з системою необхідно створити адміністративний доступ. Адміністратор має створити першого користувача з правами суперкористувача та ініціалізувати необхідні довідники, такі як список ролей. Це також виконується через інтерактивну консоль всередині контейнера за допомогою запуску спеціального скрипта ініціалізації, який перевіряє наявність базових ролей (Admin, Manager, Worker, User) і створює їх за необхідності, після чого реєструє обліковий запис адміністратора з безпечно хешованим паролем.

Завершальним етапом є експлуатація та моніторинг. Веб-портал стає доступним через веб-браузер за адресою localhost. Для управління життєвим циклом додатка використовуються стандартні команди: перегляд журналів контейнера дозволяє діагностувати помилки або аналізувати продуктивність ML-моделей, а команда down коректно завершує роботу сервісів та відключає мережу, зберігаючи при цьому дані у персистентних томах. Такий підхід до розгортання забезпечує швидкий старт, надійну ізоляцію процесів та створює передумови для легкої масштабованості системи в майбутньому.

3.6 Експериментальні дослідження

Метою проведення експериментальних досліджень була перевірка працездатності розробленого програмного засобу та вимірювання його часових характеристик у реальних умовах експлуатації. Основний акцент було зроблено на дослідженні швидкодії ключових інтелектуальних компонентів системи, а саме ієрархічної моделі класифікації тексту та алгоритму автоматичного розподілу завдань, оскільки саме вони визначають ефективність автоматизації процесів служби підтримки.

Для забезпечення високої точності вимірювань було використано інструментарій стандартної бібліотеки мови програмування Python, зокрема функцію для доступу до системного таймера високої роздільної здатності.

Методика експерименту полягала в інструменталізації програмного коду обробника створення нової заявки шляхом додавання контрольних точок для фіксації часу на початку та в кінці кожного ключового етапу обробки даних. Це дозволило отримати деталізовану картину витрат часу на різні підпроцеси та виявити потенційні вузькі місця системи.

Дослідження передбачало проведення серії з 50 експериментів, що полягали у послідовному створенні нових заявок через веб-інтерфейс системи. Вибірка вхідних даних включала текстові описи проблем різної довжини та тематики, що охоплювали всі підтримувані категорії ІТ-завдань, а також приклади нерелевантних запитів для перевірки механізму фільтрації. Така кількість ітерацій дозволила нівелювати вплив випадкових факторів операційної системи та отримати статистично значущі результати для аналізу середніх показників продуктивності.

У ході кожного експерименту фіксувалися чотири основні часові метрики. Першою метрикою був час роботи підсистеми обробки природної мови, який включав завантаження вхідного тексту, його токенізацію та послідовну роботу двох моделей: бінарного класифікатора для визначення релевантності запиту та мультикласового класифікатора для визначення типу проблеми. Другою метрикою був час виконання алгоритму розподілу завдань, що охоплював виконання складних запитів до бази даних для пошуку компетентних працівників, розрахунок їхнього рейтингу на основі навантаження та експертизи, а також вибір оптимального виконавця або прийняття рішення про ескалацію. Третьою метрикою був час фіксації транзакції у базі даних, що відображав швидкість збереження створеного завдання та записів про його призначення. Четвертою метрикою був загальний час обробки HTTP-запиту від моменту отримання даних з форми до моменту перенаправлення користувача на сторінку підтвердження.

Окрему увагу під час досліджень було приділено аналізу різниці між першим запуску системи, так званим холодним стартом, та подальшою роботою у штатному режимі. Це дозволило оцінити накладні витрати на первинну ініціалізацію моделей машинного навчання та кешування структур даних. Отримані в результаті експериментів дані були залогійовані з мікросекундною точністю та використані для подальшого розрахунку ефективності запропонованого рішення, аналіз якої наведено у наступному підрозділі.

3.7 Оцінка ефективності запропонованого рішення

На основі проведених експериментальних досліджень було виконано детальний аналіз часових характеристик роботи розробленого програмного засобу. Результати вимірювань, отримані в ході обробки 50 тестових запитів, дозволяють зробити висновки щодо продуктивності системи в цілому та її окремих компонентів. Результати дослідження представлені у Додатку Х. Результати дослідження також наведені на графіку на рисунку 3.27.

Час обробки запиту

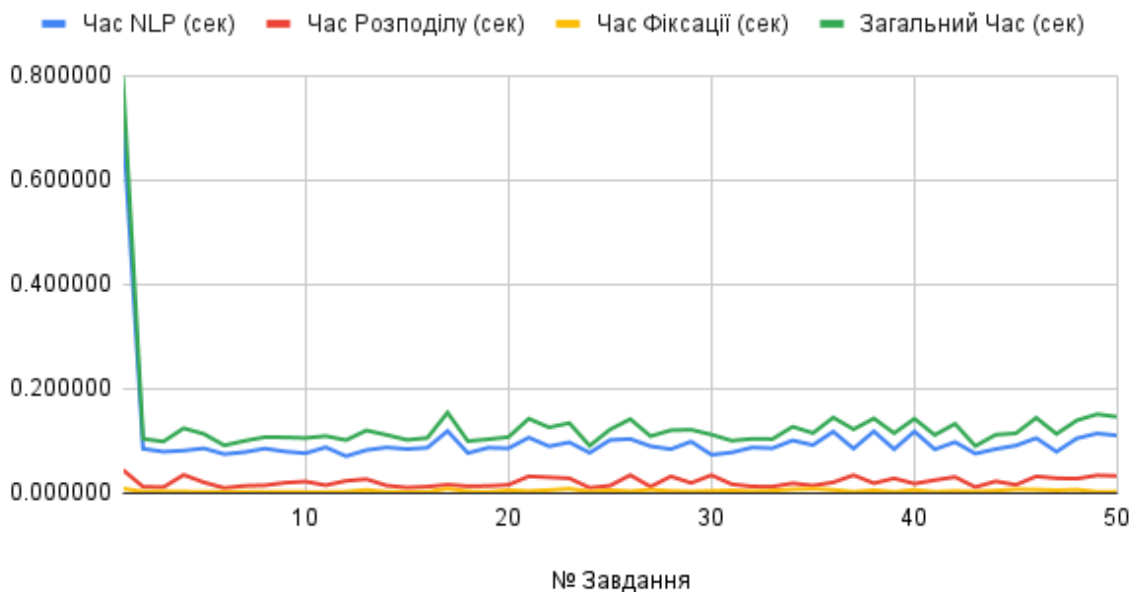


Рисунок 3.27 – Результати оцінки часу обробки запиту

Аналіз першого запиту до системи виявив явище так званого холодного старту, де загальний час обробки склав приблизно 0.8 секунди. Більшу частку цього часу, близько 0.71 секунди, зайняла робота підсистеми обробки природної мови. Це пояснюється необхідністю первинної ініціалізації моделей машинного навчання, завантаження їх у оперативну пам'ять та підготовки до виконання визначення заявки. Такий час затримки є одноразовим і допустимим для веб-орієнтованих систем, оскільки він виникає лише при першому зверненні після запуску сервера.

Подальші 49 експериментів продемонстрували стабілізацію роботи системи у режимі гарячого старту. Середній загальний час обробки заявки знизився до рівня 0.11 – 0.12 секунди, що свідчить про високу чутливість системи та миттєву реакцію на дії користувача. Детальний розгляд складових цього часу показує, що найбільш ресурсоємним етапом залишається класифікація тексту, яка займає в середньому від 0.07 до 0.11 секунди. Це підтверджує, що використання легкої архітектури DistilBERT було виправданим рішенням, яке забезпечило баланс між точністю розпізнавання та швидкістю роботи.

Алгоритм розподілу завдань показав високу ефективність, витрачаючи на пошук оптимального виконавця в середньому від 0.01 до 0.035 секунди. Така швидкість досягається завдяки оптимізованим запитам до бази даних та ефективній логіці фільтрації кандидатів. Час фіксації змін у базі даних виявився незначним і стабільно становив менше 0.01 секунди, що свідчить про відсутність проблем з продуктивністю на рівні системи зберігання даних.

Результати також підтвердили коректність роботи логіки системи в різних сценаріях. У випадках, коли алгоритм не знаходив доступних виконавців, час обробки залишався в межах норми, а система коректно виконувала ескалацію завдання на менеджера. Варіативність часу виконання

NLP-модуля у деяких випадках, наприклад, зростання до 0.11 секунди у 17-му експерименті, може бути пояснена різною довжиною вхідного тексту або тимчасовим навантаженням на процесор, проте ці відхилення не є критичними.

Автоматизація процесів класифікації та розподілу дозволила скоротити час маршрутизації заявки до часток секунди, що є недосяжним показником для ручної обробки менеджером. Це підтверджує практичну цінність запропонованого рішення та досягнення мети дисертаційної роботи щодо підвищення ефективності служби IT-підтримки.

Висновки до розділу

У третьому розділі магістерської дисертації було детально розглянуто етапи проектування та програмної реалізації вебпорталу для автоматизації розподілу навантаження служби IT-підтримки. На основі попереднього аналізу предметної області було сформульовано чіткі функціональні та нефункціональні вимоги до системи, які враховують розмежування прав доступу для ролей адміністратора, менеджера, працівника та користувача. Розроблена архітектура додатка побудована за шаблоном MVC, що забезпечило модульність системи та успішну інтеграцію навчених моделей машинного навчання для класифікації тексту.

В ході роботи було створено інтуїтивно зрозумілий графічний інтерфейс користувача та реалізовано ключову бізнес-логіку, включаючи автоматичну маршрутизацію заявок та механізми ескалації завдань. Проведені експериментальні дослідження підтвердили працездатність розробленого рішення та високу швидкість його роботи. Результати тестування показали, що середній час повної обробки запиту в усталеному режимі складає близько 0,11–0,12 секунди, що значно перевершує швидкість ручної обробки та повністю відповідає вимогам до швидкодії систем реального часу.

4 МАРКЕТИНГОВИЙ АНАЛІЗ СТАРТАП-ПРОЄКТУ

4.1 Опис ідеї проєкту

Таблиця 4.1 — Опис ідеї стартап-проєкту

Зміст ідеї	Напрямки застосування	Вигоди для користувача
Створення інтелектуальної системи автоматизації служби підтримки, яка використовує NLP для класифікації заявок та алгоритми для розподілу завдань.	ІТ-відділи середніх та великих підприємств. Аутсорсингові компанії, що надають послуги підтримки.	Скорочення часу обробки заявки (з хвилин до секунд); Зменшення навантаження на менеджерів (відсутність ручного розподілу); Підвищення точності призначення виконавців (врахування компетенцій).
Використання ієрархічної моделі класифікації для роботи з українською мовою та фільтрації нерелевантних запитів.	Локальні українські компанії.	Висока точність розуміння україномовних запитів. Зменшення кількості помилкових спрацювань на нецільові запити (OOD).

Таблиця 4.2 — Опис ідеї стартап-проекту

№	Техніко-економічні характеристики ідеї	Продукція конкурентів			W (слабка сторона)	N (нейтр. сторона)	S (сильна сторона)
		Мій проект	Jira	Zendesk			
1	Автоматична класифікація на основі NLP	+	-	-			+
2	Підтримка української мови (морфологія, контекст)	+	+	-			+
3	Автоматичний розподіл за компетенціями та навантаженням	+	-	-			+
4	Фільтрація нерелевантних (OOD) запитів	+	-	-			+
5	Інтеграція з існуючими екосистемами	-	+	+	+		
6	Впізнаваність бренду	-	+	+	+		

4.2 Технологічний аудит ідеї проекту

Таблиця 4.3 — Технологічна здійсненність ідеї проекту

№	Ідея проекту	Технології її реалізації	Наявність технологій	Доступність технологій
1	Класифікація тексту (NLP)	Python, Transformers (Hugging Face), PyTorch, DistilBERT	Наявна	Доступна (Open Source)
2	Веб-додаток та API	Python, Flask, REST API	Наявна	Доступна
3	Зберігання даних	PostgreSQL, SQLAlchemy	Наявна	Доступна

Завершення таблиці 4.3

4	Розгортання та контейнеризація	Docker, Docker Compose	Наявна	Доступна
<i>Обрана технологія реалізації ідеї проекту: Python, Flask, Transformers, PostgreSQL</i>				

Висновок: технологічна реалізація продукту – можлива, вибрано технології Python (Flask + Transformers) та PostgreSQL.

4.3 Аналіз ринкових можливостей запуску стартап-проєкту

Таблиця 4.4. — Попередня характеристика потенційного ринку

№	Показники стану ринку	Характеристика
1	Кількість головних гравців, од	5-7 (Atlassian, ServiceNow, Zendesk, Freshworks, Zoho)
2	Загальний обсяг продаж, грн./ум.од	> 8 млрд. дол. (ринок ITSM)
3	Динаміка ринку	Зростаючий (CAGR ~15% через цифровізацію бізнесу)
4	Наявність обмежень для входу	Середня (висока конкуренція, але низький поріг входу для нішевих рішень)
5	Специфічні вимоги до стандартизації та сертифікації	GDPR (захист даних), ISO 20000 (для ITSM)
6	Середня норма рентабельності в галузі або по ринку, %	20-30% (SaaS рішення)

Висновок: враховуючи кількість головних гравців по ринку, зростаючу динаміку ринку, невелику кількість конкурентів та середню норму рентабельності можна зробити висновок, що на даний момент, ринок для входження стартап-продукту є привабливим.

Таблиця 4.5. — Характеристика потенційних клієнтів стартап-проекту

№	Потреба, що формує ринок	Цільова аудиторія	Відмінності у поведінці цільових груп клієнтів	Вимоги споживачів до товару
1	Автоматизація рутинних процесів підтримки	ІТ-компанії середнього розміру (50-500 співробітників)	Цінують гнучкість та швидкість впровадження	Швидка інтеграція, низька ціна, API; Надійність, підтримка, звітність; Локалізація, захист даних, україномовний інтерфейс.
2	Зниження витрат на персонал	Великі підприємства (не ІТ сектор: retail, логістика)	Консервативні, вимагають стабільності	
3	Обробка запитів державною мовою	Державні установи України	Вимагають відповідності законодавству	

Таблиця 4.6. — Фактори загроз

№	Фактор	Зміст загрози	Можлива реакція компанії
1	Технологічний	Швидкий розвиток LLM (ChatGPT) у конкурентів	Постійне донавчання власних моделей, використання гібридних підходів (як у проєкті).
2	Конкурентний	Демпінг цін з боку великих гравців	Фокусування на ніші (український ринок), краща якість локалізації.
3	Економічний	Скорочення бюджетів компаній на ІТ	Пропонування моделі оплати "за результат" або Freemium версії.

Таблиця 4.7. — Фактори можливостей

№	Фактор	Зміст можливості	Можлива реакція компанії
1	Зростання попиту на AI	Компанії шукають AI-рішення для оптимізації	Акцентування в маркетингу на "Intelligent Assignment" та AI-складовій.
2	Локалізація	Відсутність якісних україномовних NLP-рішень у гігантів	Позиціонування як "Найкраще рішення для українського бізнесу".
3	Хмарні технології	Здешевлення хостингу та ресурсів	Використання хмарної SaaS моделі для легкого масштабування.

Таблиця 4.8. — Ступеневий аналіз конкуренції на ринку

№	Особливості конкурентного середовища	В чому проявляється дана характеристика	Вплив на діяльність підприємства (можливі дії компанії, щоб бути конкурентоспроможною)
1	Тип конкуренції: монополістична	Товар від кожної компанії на ринку, являється недосконалим замінником товару, реалізованого іншими фірмами; На ринку є умови для входу та виходу; Ціна корелює між суперниками;	Розробка продукту з характеристиками, які покривають сфери вживання що не покривають інші товари-замінники; Кореляція цін у відповідності до товарів замінників; Різні типи ліцензій.

Завершення таблиці 4.8

2	Рівень конкурентної боротьби: світовий	Всі продукти замітники розроблялись інтернаціональними командами з різних куточків світу, продукти не належать до певної держави, а належать команді розробників	Вихід на ринок збуту продукту з клієнто-необхідною функціональністю; Налагодження маркетингу на основних Інтернет ресурсах задля охоплення великої кількості потенційних користувачів; Надання бета-версій продукту.
3	Галузева ознака: загальногалузева	Даний тип продукту може використовуватися у сфері B2B ПЗ (SaaS)	Надання зручного, інтуїтивно зрозумілого інтерфейсу; Підтримка всім відомих методів взаємодії з середовищем розробки; Наявність документації та он-лайн підтримки.

Таблиця 4.9. — Аналіз конкуренції в галузі за М. Портером

Складові аналізу	Прямі конкуренти в галузі	Потенційні конкуренти	Постачальники	Клієнти	Товари-замінники
	Jira SM, Zendesk, Freshservice (Сильні бренди, широкий функціонал).	Нові стартапи на базі OpenAI API, внутрішні розробки компаній.	Провайдери хмарних послуг (AWS, Azure, Hetzner). Вплив низький.	Середній та великий бізнес. Мають високу силу торгу, вимагають якості.	Telegram-боти, Excel-таблиці, електронна пошта (для малих компаній).

Проаналізувавши можливості роботи на ринку з огляду на конкурентну ситуацію можна зробити висновок: оскільки кожний з існуючих продуктів не

впливає у великій мірі на поточну ситуацію на ринку в цілому, кожний з існуючих продуктів має свою специфічну сферу використання та свої позитивні та негативні сторони щодо рішення певних типів задач, то робота та вихід на даний ринок є можливою і реалізованою задачею [10].

Для виходу на ринок продукт повинен мати функціонал що відсутній у продуктів-аналогів, повинен задовольняти потреби користувачів, мати необхідний та достатній функціонал з конфігурування, підтримку зі сторони розробників та можливість розробки спеціального функціоналу за відповідною ліцензією.

Таблиця 4.10. — Обґрунтування факторів конкурентоспроможності

№	Фактор конкурентоспроможності	Обґрунтування
1	Точність класифікації (Укр)	Використання донавченої моделі DistilBERT саме на українських даних.
2	Інтелектуальний розподіл	Алгоритм враховує не тільки тип, а й навантаження та скіли (чого немає в базових версіях конкурентів).
3	Ціна	Нижча вартість порівняно з Enterprise-ліцензіями західних вендорів.
4	Швидкість розгортання	Docker-контейнеризація дозволяє розгорнути систему за лічені хвилини on-premise.
5	Функціональність (загальна)	Засіб має весь функціонал для розподілення навантаження
6	Зручність інтерфейсу	Портал має зручний інтерфейс для створення та виконання заявок
7	Технічна підтримка	Всі користувачі можуть зв'язатись з адміністраторами для вирішення технічних питань

Таблиця 4.11. — Порівняльний аналіз сильних та слабких сторін системи кешування мало змінних даних

№	Фактор конкурентоспроможності	Бали 1-20	Рейтинг товарів-конкурентів у порівнянні з запропонованим						
			-3	-2	-1	0	+1	+2	+3
1	Точність класифікації (Укр)								+
2	Інтелектуальний розподіл							+	
3	Ціна						+		
4	Швидкість розгортання					+			
5	Функціональність (загальна)				+				
6	Зручність інтерфейсу						+		
7	Технічна підтримка						+		

Таблиця 4.12. — SWOT аналіз стартап-проєкту

Сильні сторони (S): — Спеціалізована модель для укр. мови; — Ефективний алгоритм розподілу навантаження; — Низька вартість володіння; — Гнучкість розгортання.	Слабкі сторони (W): — Обмежений загальний функціонал (порівняно з Jira); — Невідомий бренд; — Відсутність мобільного додатку; — Невелика команда розробки.
Можливості (O): — Вихід на ринок держустанов; — Інтеграція з месенджерами (Telegram/Viber); — Розширення набору мов.	Загрози (T): — Поява україномовних моделей у глобальних конкурентів; — Зміна технологічних трендів (перехід на no-code).

Таблиця 4.13. — Альтернативи ринкового впровадження стартап-проєкту

№	Альтернатива (орієнтовний комплекс заходів) ринкової поведінки	Ймовірність отримання ресурсів	Строки реалізації
1	Безкоштовне надання певного функціоналу у користування споживачам на обмежений термін	Головний ресурс – люди, даний ресурс - наявний	2-3 місяці
2	Реклама	Залучення власних коштів для реклами товару	1-2 місяці
3	Написання статей та опис товару на відомих ресурсах	Головний ресурс – час, даний ресурс - наявний	2-3 тижні
4	Презентація товару на хакатонах й інших ІТ заходах	Ресурс – час та гроші для участі, наявні	1-3 місяці

4.4 Розроблення ринкової стратегії проєкту

Таблиця 4.14. — Вибір цільових груп потенційних споживачів

№	Опис профілю цільової групи потенційних клієнтів	Готовність споживачів сприйняти продукт	Орієнтовний попит в межах цільової групи (сегменту)	Інтенсивність конкуренції в сегменті	Простота входу у сегмент
1	ІТ-компанії (Software Houses)	Висока	Середній	Висока	Середня
2	Великий рітейл / Логістика (внутрішні ІТ відділи)	Середня	Високий	Середня	Середня

Закінчення таблиці 4.15

Які цільові групи обрано: Внутрішні ІТ-відділи середніх та великих українських підприємств (непрофільний ІТ-сектор), які потребують оптимізації процесів без купівлі наддорогих ліцензій.

Відповідно до проведеного аналізу можна зробити висновок, що підходящою цільовою групою для розповсюдження даного програмного продукту є працівники ІТ сфери, ІТ компанії в цілому та будь-які підприємства котрі використовують програмні продукти побудовані на мові програмування Java, та використовують реляційні бази даних. Відповідно до стратегії охоплення ринку збуту товару обрано стратегію масового маркетингу, оскільки для підприємств, ІТ працівників та ІТ компаній у цілому надається стандартизований продукт з можливістю розширення функціональності за домовленістю (відповідно до ліцензії).

Таблиця 4.15. — Визначення базової стратегії розвитку

Обрана альтернатива розвитку проєкту	Стратегія охоплення ринку	Ключові конкурентоспроможні позиції відповідно до обраної альтернативи	Базова стратегія розвитку
Розробка спеціалізованого українського продукту з фокусом на NLP та автоматизацію	Нішова спеціалізація (фокусування)	Висока точність обробки української мови; Інтегрований алгоритм розподілу; Адаптація під місцеві бізнес-процеси.	Стратегія спеціалізації (фокусування)

Таблиця 4.16. — Визначення базової стратегії конкурентної поведінки

Чи є проєкт «першопрохідцем» на ринку	Чи буде компанія шукати нових споживачів, або забирати існуючих у конкурентів?	Чи буде компанія копіювати основні характеристики товару конкурента, які?	Стратегія конкурентної поведінки
Ні, на ринку існують глобальні гравці (Jira, Zendesk)	Основний фокус на нових клієнтах, які ще не автоматизували процеси, та переманюванні тих, кого не влаштовує ціна/локалізація гігантів.	Так, базовий функціонал Service Desk (створення, облік, статуси заявок). Але з додаванням унікального AI-модуля.	Стратегія нішера (заповнення ніші україномовних AI-рішень для Service Desk)

Таблиця 4.17. — Визначення стратегії позиціонування

№	Вимоги до товару цільової аудиторії	Базова стратегія розвитку	Ключові конкурентоспроможні позиції власного стартап-проєкту	Вибір асоціацій, які мають сформувати комплексну позицію власного проєкту
1	Швидкість та точність обробки	Спеціалізація	Автоматична класифікація за < 1 сек	"Миттєве вирішення"
2	Розуміння специфіки мови	Спеціалізація	Якісна робота з українською мовою	"Своє, рідне, зрозуміле"
3	Справедливий розподіл	Спеціалізація	Алгоритм балансування навантаження	"Ефективність команди"

Відповідно до проведеного аналізу можна зробити висновок, що стартап-компанія вибирає як базову стратегію розвитку – стратегію диференціації, як базову стратегію конкурентної поведінки – стратегію заняття конкурентної ніші.

4.5 Розроблення маркетингової програми програми стартап-проекту

Таблиця 4.18. — Визначення ключових переваг концепції потенційного товару

№	Потреба	Вигода, яку пропонує товар	Ключові переваги перед конкурентами (існуючі або такі, що потрібно створити)
1	Автоматизація рутинних дій менеджера	Звільнення часу на стратегічні задачі	Повністю автоматичний цикл: Класифікація -> Пріоритет -> Виконавець (у конкурентів часто потребує складного налаштування)
2	Ефективне використання ресурсів команди	Рівномірне навантаження на працівників	Динамічний алгоритм розподілу, що враховує поточну зайнятість (Smart Assignment)
3	Робота з україномовним контентом	Точне розуміння суті звернень без перекладу	Спеціалізована NLP-модель, донавчена на українських даних
4	Зниження витрат	Доступна автоматизація	Відсутність дорогої ліцензії за кожне "розумне" правило

Таблиця 4.19. — Опис трьох рівнів моделі товару

Рівні товару	Сутність та складові		
1. Товар за задумом	Програмний засіб для інтелектуальної автоматизації служби підтримки, що вирішує проблему ручного розподілу заявок		
2. Товар у реальному виконанні	Властивості/характеристики	М/Нм	Вр/Тх/Тл/Е/Ор
	Автоматична класифікація заявок (NLP)	Нм	Вр
	Автоматичний розподіл (Algorithm)	Нм	Вр
	Веб-інтерфейс (User Dashboard)	Нм	Е

Завершення таблиці 4.19

	Панель адміністратора/менеджера	Нм	Е
	База даних PostgreSQL	Нм	Тх
	Використання Python/Flask	Нм	Тх
	Моделі Transformers (AI)	Нм	Тл
3. Товар із підкріпленням	До продажу: безкоштовна демо-версія, презентація можливостей, консультація щодо впровадження.		
	Після продажу: інструкції з розгортання (Docker), технічна підтримка, оновлення моделей, виправлення багів.		
За рахунок чого потенційний товар буде захищено від копіювання: реєстрація авторського права на комп'ютерну програму, "know-how" у налаштуванні моделей (fine-tuning).			

Таблиця 4.20. — Визначення меж встановлення ціни

Рівень цін на товари-замінники	Рівень цін на товари-аналоги	Рівень доходів цільової групи споживачів	Верхня та нижня межі встановлення ціни на товар/послугу
Безкоштовно (Excel, пошта, месенджери)	Високий (\$50-150/міс за Enterprise плани з AI)	Середній (український середній бізнес)	Нижня: Безкоштовно (Open Source / Self-hosted) Верхня: \$10-20/агент/міс (за SaaS версію з підтримкою)

Таблиця 4.21. — Формування системи збуту

Специфіка закупівельної поведінки цільових клієнтів	Функції збуту, які має виконувати постачальник товару	Глибина каналу збуту	Оптимальна система збуту
B2B клієнти приймають рішення раціонально, довго тестують, потребують погодження бюджету.	Демонстрація можливостей; Допомога в інтеграції (Onboarding); Надання тестового періоду.	Канал нульового рівня (прямий продаж)	Прямі продажі через власний веб-сайт та контакт з ЛПР (особами, що приймають рішення)

Таблиця 4.22. — Концепція маркетингових комунікацій

№	Специфіка поведінки цільових клієнтів	Канали комунікацій, якими користуються цільові клієнти	Ключові позиції, обрані для позиціонування	Завдання рекламного повідомлення	Концепція рекламного звернення
1	ІТ-компанії: Шукають конкретні технічні рішення, цінують час, приймають рішення на основі функціоналу та ціни. Активно вивчають огляди та кейси.	Професійні спільноти (DOU, LinkedIn); Технічні блоги (Medium, Dev.to); Рекомендації колег.	"Розумна автоматизація"; "Економія часу команди"	Показати, як автоматичний розподіл завдань звільняє менеджерів від рутини та пришвидшує роботу.	Забезпечте ефективність вашої ІТ-підтримки. Автоматизуйте розподіл завдань за допомогою інтелектуальних NLP-алгоритмів.

Як результат було створено ринкову (маркетингову) програму, що включає в себе визначення ключових переваг концепції потенційного товару, опис моделі товару, визначення меж встановлення ціни, формування системи збуту та концепцію маркетингових комунікацій.

Висновки до розділу

В четвертому розділі описано стратегії та підходи з розроблення стартап-проєкту, визначено наявність попиту, динаміку та рентабельність роботи ринку, як висновок було вказано що існує можливість ринкової комерціалізації проєкту. Розглянувши потенційні групи клієнтів, бар'єри входження, стан конкуренції та конкурентоспроможність проєкту було встановлено що проєкт є перспективним. Розглянуто та вибрано альтернативу впровадження стартап-проєкту та доведено доцільність подальшої імплементації проєкту.

ВИСНОВКИ

У ході виконання магістерської дисертації було розглянуто питання пов'язані з автоматизацією процесів у службах IT-підтримки, зокрема проблематику ручної класифікації текстових заявок, обробки неструктурованих даних українською мовою та неоптимального розподілу навантаження між операторами.

На основі даних, отриманих в процесі аналізу, сформульовано задачу розробки інтелектуального програмного засобу, здатного автоматично визначати тип проблеми з тексту заявки, відсіювати нерелевантні запити та призначати відповідного виконавця з урахуванням його компетенцій та поточного навантаження.

Для розробки вебпорталу та моделей машинного навчання було використано мову програмування Python, мікрофреймворк Flask, систему управління базами даних PostgreSQL, бібліотеку SQLAlchemy та бібліотеку Transformers від Hugging Face. Все перераховане є широко вживаними та безкоштовними інструментами, що дозволить знизити вартість впровадження системи, забезпечити її надійність та легку масштабованість.

Розроблена модель класифікації текстових звернень побудована за ієрархічним принципом на базі архітектури DistilBERT та включає етап бінарної фільтрації для відсіювання OOD-запитів і етап мультикласової класифікації для визначення категорії проблеми.

Розроблено програмне забезпечення у вигляді вебпорталу служби підтримки, яке автоматизує повний життєвий цикл заявки, реалізує рольову модель доступу (користувач, працівник, менеджер, адміністратор) та інтегрує алгоритми NLP для маршрутизації завдань.

Проведений маркетинговий аналіз стартап-проєкту. Проведено опис ідеї проєкту, технологічний аудит ідеї проєкту, аналіз ринкових можливостей

запуску стартап-проєкту. Розроблено ринкову стратегію проєкту та маркетингову програму стартап-проєкту.

Результати роботи над магістерською дисертацією пройшли апробацію на науково-практичній конференції молодих вчених та студентів «Інженерія програмного забезпечення і передові інформаційні технології» (SoftTech-2025).

Наукова новизна одержаних результатів магістерської дисертації полягає у набутті подальшого розвитку застосування NLP-моделей для автоматизації процесів ІТ-підтримки шляхом інтеграції ієрархічної класифікації з алгоритмом розподілу завдань, що враховує компетенції та навантаження виконавців.

Практична значимість одержаних результатів полягає у розробці програмного засобу розподілу, що дозволить скоротити час обробки заявок, тим самим збільшити швидкість реагування на проблеми та збільшити ефективність роботи організації. Розроблений модуль можна буде використати у роботі служб ІТ підтримки різних компаній.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

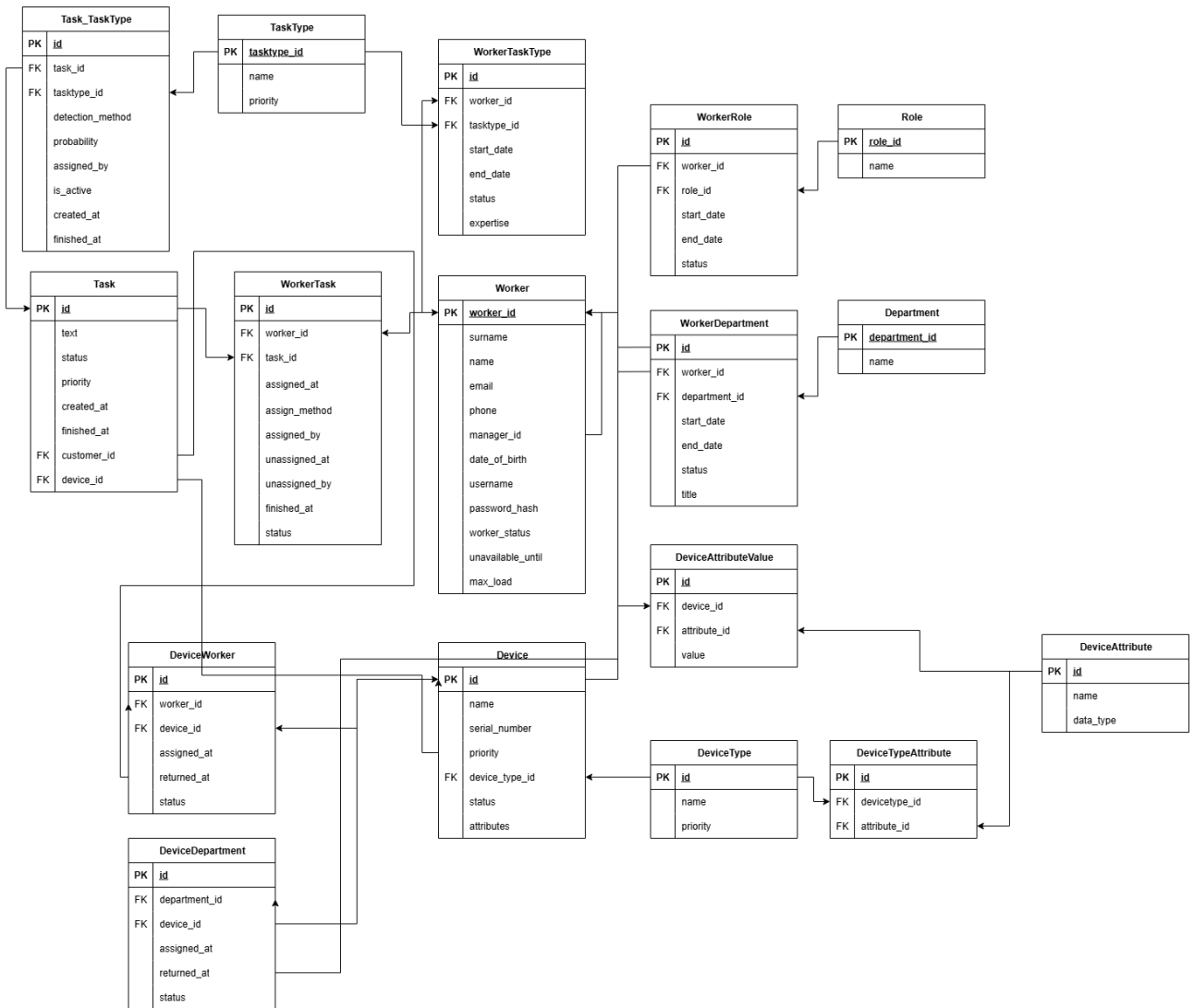
- 1) Self-service: Do customers want to help themselves? (infographic). Zendesk. (2023, May 17). <https://www.zendesk.com/blog/searching-for-self-service/>
- 2) Paramesh, J., & Ravi, V. (2019). Classifying the Unstructured IT Service Desk Tickets Using Ensemble of Classifiers. IEEE International Conference on Computational Intelligence in Data Science (ICCIDS). <https://doi.org/10.1109/ICCIDS.2019.8862130>
- 3) Silva, L., et al. (2018). Text classification for IT service desk tickets. Proceedings of the IEEE International Conference on Information Reuse and Integration (IRI).
- 4) Brynjolfsson, E., Li, D., & Raymond, L. R. (2023). Generative AI at Work. National Bureau of Economic Research Working Paper Series, No. 31161. <https://doi.org/10.3386/w31161>
- 5) Negahban, A., & Smith, J. S. (2014). Simulation for manufacturing system design and operation: A comprehensive review. International Journal of Production Economics, 142, 381-402.
- 6) Djamila Romaissa Beddiar, Md Saroar Jahan, Mourad Oussalah. (2021). Data expansion using back translation and paraphrasing for hate speech detection, Online Social Networks and Media, Volume 24,. <https://doi.org/10.1016/j.osnem.2021.100153>.
- 7) Olanrewaju, T. (2024, December 11). How django's MVT Architecture Works: A deep dive into models, views, and templates. freeCodeCamp.org. <https://www.freecodecamp.org/news/how-django-mvt-architecture-works/>
- 8) Modular applications with blueprints¶. Modular Applications with Blueprints - Flask Documentation (3.1.x). (n.d.). <https://flask.palletsprojects.com/en/stable/blueprints/>
- 9) What is a container?. Docker. (2025, November 24). <https://www.docker.com/resources/what-container/>

- 10) Розроблення стартап-проєкту [Електронний ресурс] : Методичні рекомендації до виконання розділу магістерських дисертацій для студентів інженерних спеціальностей / За заг. ред. О.А. Гавриша. – Київ : НТУУ «КПІ», 2016. – 28 с.

ДОДАТКИ

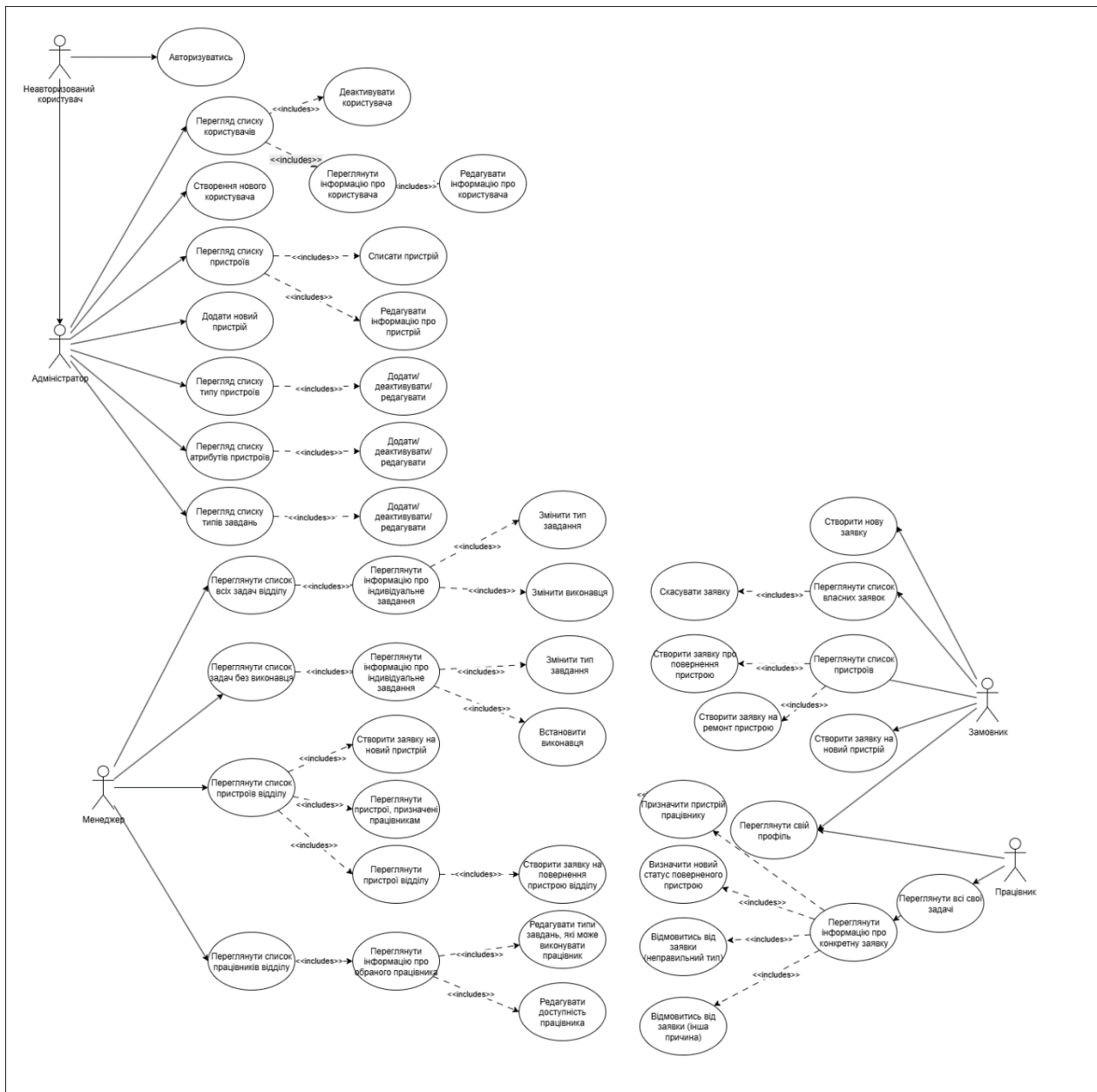
ДОДАТОК А

Схема бази даних



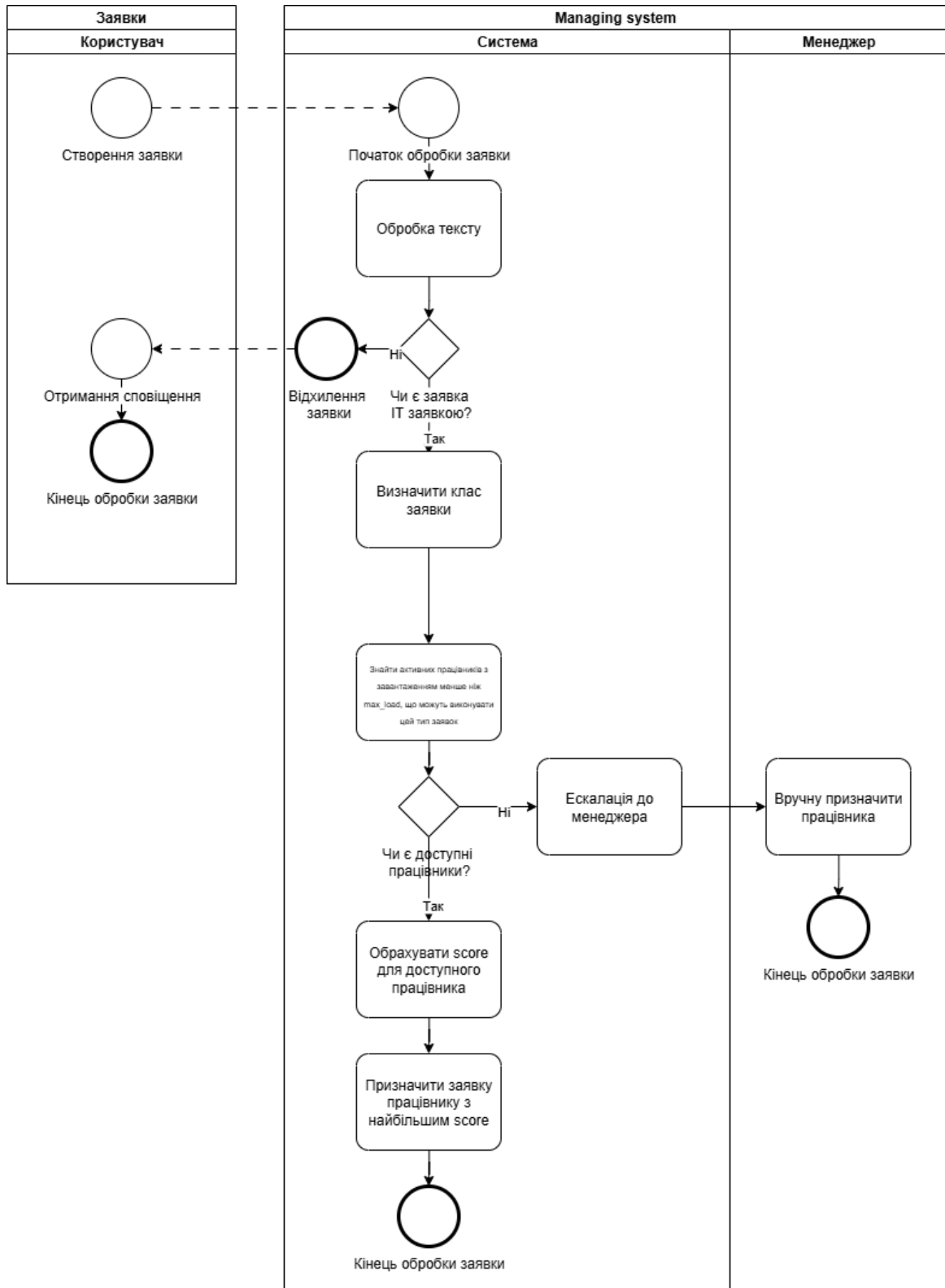
ДОДАТОК Б

UML діаграма класів



ДОДАТОК В

Алгоритм обробки працівників



ДОДАТОК Г

Лістинг коду

Повний лістинг коду розташований за посиланням:

<https://github.com/marynatsuk/SupportServiceLoadBalancing>

Файл app.py

Реалізація створення застосунку фреймворку Flask

```
from app import create_app
app = create_app()
if __name__ == '__main__':
    app.run(debug=True)
```

Файл __init__.py

Реалізація ініціалізації застосунку, ініціалізація шаблонів для створення сторінок та функціоналу авторизації користувачів

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager
from flask_migrate import Migrate
from apscheduler.schedulers.background import BackgroundScheduler
from os import path
db = SQLAlchemy()
migrate = Migrate()
login_manager = LoginManager()

def create_app():
    app = Flask(__name__)

    app.config['SECRET_KEY'] = 'lqiefuvnl sdfvbuIU34FJVF'
    app.config['SQLALCHEMY_DATABASE_URI'] = 'postgres://postgres:12345@localhost:5432/itportal'
    app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
    app.config['SQLALCHEMY_ECHO'] = False

    db.init_app(app)
    migrate.init_app(app, db)
    login_manager.init_app(app)

    login_manager.login_view = 'auth.login'
    login_manager.login_message = "Будь ласка, увійдіть, щоб отримати доступ до цієї сторінки."
    login_manager.login_message_category = "info"

    from .models import Worker
    from .admin_views import admin_views
    from .auth import auth
```

```

from .manager_views import manager_views
from .user_views import user_views
from .worker_views import worker_views
from .asset_views import asset_views
from .scheduler import check_and_update_vacation_status

app.register_blueprint(auth, url_prefix='/')
app.register_blueprint(admin_views, url_prefix='/admin')
app.register_blueprint(manager_views, url_prefix='/manager')
app.register_blueprint(user_views, url_prefix='/user')
app.register_blueprint(worker_views, url_prefix='/worker')
app.register_blueprint(asset_views, url_prefix='/assets')

@login_manager.user_loader
def load_user(user_id):
    return Worker.query.get(int(user_id))

with app.app_context():
    db.create_all()

if not hasattr(app, 'scheduler'):
    app.scheduler = BackgroundScheduler(daemon=True)
    app.scheduler.add_job(check_and_update_vacation_status, 'cron', hour=3,
minute=0, args=[app])
    app.scheduler.start()
    print("Планувальник завдань запущено.")

return app

```

Файл status_utils.py

Реалізація алгоритму визначення типу завдання з його тексту

```

from flask import request, redirect, url_for, flash, render_template
from datetime import datetime
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

# --- Багатокласовий класифікатор ---
expert_model_path = "app/my_final_model"
tokenizer_expert = AutoTokenizer.from_pretrained(expert_model_path)
model_expert = AutoModelForSequenceClassification.from_pretrained(expert_model_path)

# --- Бінарний класифікатор ---
binary_model_path = "app/binary_model"
tokenizer_bin = AutoTokenizer.from_pretrained(binary_model_path)
model_bin = AutoModelForSequenceClassification.from_pretrained(binary_model_path)

# --- Налаштування пристрою ---
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model_expert = model_expert.to(device)
model_bin = model_bin.to(device)
model_expert.eval()

```

```

model_bin.eval()

# --- Мапа класів для мультикласового класифікатора---
label_map_expert = {
    0: "Network issues",
    1: "Software Management",
    2: "Account Management",
    3: "Password issues",
    4: "Security incidents",
    5: "File and Storage management",
    6: "Application Management"
}

def predict_binary(text):
    inputs = tokenizer_bin(text, return_tensors="pt", truncation=True,
padding=True).to(device)
    with torch.no_grad():
        outputs = model_bin(**inputs)
        logits = outputs.logits
        probs = torch.softmax(logits, dim=1)
        predicted_class_id = torch.argmax(probs, dim=1).item()
        confidence = probs[0][predicted_class_id].item()
    return predicted_class_id, confidence

def predict_expert(text):
    inputs = tokenizer_expert(text, return_tensors="pt", truncation=True,
padding=True).to(device)
    with torch.no_grad():
        outputs = model_expert(**inputs)
        logits = outputs.logits
        probs = torch.softmax(logits, dim=1)
        predicted_class_id = torch.argmax(probs, dim=1).item()
        predicted_label = label_map_expert[predicted_class_id]
        confidence = probs[0][predicted_class_id].item()
    return predicted_class_id, predicted_label, confidence

```

Файл manager_views.py

Реалізація логіки роботи менеджера

```

from flask import Blueprint, render_template, request, flash, redirect, url_for,
jsonify
from flask_login import login_required, current_user
from .decorators import role_required
import traceback

from .worker_assign import choose_worker

from .models import (
    Worker, WorkerDepartment, WorkerTaskType,
    Task, TaskType, WorkerTask, Device, DeviceWorker, DeviceDepartment,
    DeviceType, DeviceAttribute, Task_TaskType,
    WorkerStatusEnum, HistoryStatusEnum, TaskStatusEnum, TaskDetectionMethodEnum,

```

```

        WorkerTaskStatusEnum, WorkerTaskAssignmentMethodEnum, DeviceStatusEnum,
        AssignmentStatusEnum
    )
    from . import db
    from sqlalchemy import asc, desc, func, case
    from werkzeug.security import generate_password_hash
    from datetime import datetime
    from sqlalchemy.orm import joinedload
    import re

    manager_views = Blueprint('manager_views', __name__, template_folder='manager')

    @manager_views.route('/dashboard', methods=['GET', 'POST'])
    @login_required
    @role_required('Manager')
    def manager_dashboard():

        assignments = (
            WorkerTask.query
            .join(WorkerTask.worker)
            .join(WorkerTask.task)
            .filter(Worker.manager_id == current_user.worker_id)
            .options(
                joinedload(WorkerTask.worker),
                joinedload(WorkerTask.task).joinedload(Task.customer)
            )
            .order_by(
                case(
                    (WorkerTask.status == WorkerTaskStatusEnum.ACTIVE, 0),
                    else_=1
                ).asc(),
                desc(Task.priority),
                asc(Task.created_at)
            )
            .all()
        )

        task_types_for_filter = TaskType.query.order_by(TaskType.name).all()
        status_options_for_filter = list(TaskStatusEnum)

        return render_template(
            "manager/manager_dashboard.html",
            assignments=assignments,
            task_types=task_types_for_filter,
            statuses=status_options_for_filter
        )

    @manager_views.route('/request/<int:workertask_id>', methods=['GET', 'POST'])
    @login_required
    @role_required('Manager')
    def edit_request(workertask_id):
        assignment = WorkerTask.query.get_or_404(workertask_id)
        task = assignment.task

```

```

if request.method == 'POST':
    try:
        changes_made = False
        redirect_target_id = workertask_id

        new_tasktype_id = int(request.form.get("tasktype_id"))
        active_type_record = task.active_task_type_record

        type_has_changed = False
        if active_type_record:
            if active_type_record.tasktype_id != new_tasktype_id:
                active_type_record.is_active = False
                type_has_changed = True
        elif new_tasktype_id:
            type_has_changed = True

        if type_has_changed:
            new_type_link = Task_TaskType(
                task_id=task.id,
                tasktype_id=new_tasktype_id,
                detection_method=TaskDetectionMethodEnum.MANUAL_OVERRIDE,
                assigned_by_id=current_user.id,
                is_active=True
            )
            db.session.add(new_type_link)
            changes_made = True

        new_worker_id = int(request.form.get("worker_id"))
        if assignment.worker_id != new_worker_id:
            assignment.status = WorkerTaskStatusEnum.REASSIGNED
            assignment.unassigned_at = datetime.utcnow()
            assignment.unassigned_by_id = current_user.id

            new_assignment = WorkerTask(
                task_id=task.id,
                worker_id=new_worker_id,
                assigned_by_id=current_user.id,
                assign_method=WorkerTaskAssignmentMethodEnum.MANUAL,
                status=WorkerTaskStatusEnum.ACTIVE
            )
            db.session.add(new_assignment)
            changes_made = True

        db.session.flush()
        redirect_target_id = new_assignment.id

    if changes_made:
        db.session.commit()
        flash("Зміни успішно збережено!", "success")
    else:
        flash("Ви не внесли жодних змін.", "info")

    return redirect(url_for("manager_views.edit_request",
workertask_id=redirect_target_id))

```

```

        except Exception as e:
            db.session.rollback()
            traceback.print_exc()
            flash(f"Сталася помилка: {e}", "danger")
            return redirect(url_for("manager_views.edit_request",
workertask_id=workertask_id))

    all_task_types = TaskType.query.order_by(TaskType.name).all()

    return render_template(
        "manager/edit_request.html",
        assignment=assignment,
        all_task_types=all_task_types
    )

@manager_views.route('/edit_worker/<int:user_id>', methods=['GET', 'POST'])
@login_required
@role_required('Manager')
def edit_worker_profile(user_id):
    worker_to_edit = Worker.query.get_or_404(user_id)

    if worker_to_edit.manager_id != current_user.worker_id:
        flash("Ви не маєте права редагувати профіль цього працівника.", "danger")
        return redirect(url_for('manager_views.manage_workers'))

    if request.method == 'POST':
        try:
            max_load_val = request.form.get('max_load')
            if max_load_val and max_load_val.strip():
                worker_to_edit.max_load = int(max_load_val)
            else:
                worker_to_edit.max_load = None

            selected_ids = set(request.form.getlist('tasktypes', type=int))

            current_competencies =
worker_to_edit.task_type_competencies.filter_by(status=HistoryStatusEnum.ACTIVE).
all()

            current_ids = {c.tasktype_id for c in current_competencies}

            ids_to_remove = current_ids - selected_ids
            if ids_to_remove:
                WorkerTaskType.query.filter(
                    WorkerTaskType.worker_id == user_id,
                    WorkerTaskType.tasktype_id.in_(ids_to_remove),
                    WorkerTaskType.status == HistoryStatusEnum.ACTIVE
                ).update({
                    'status': HistoryStatusEnum.INACTIVE.name,
                    'end_date': datetime.utcnow()
                }, synchronize_session=False)

            for type_id in selected_ids:
                expertise_val = int(request.form.get(f'expertise_{type_id}', 1))

```

```

    if expertise_val < 1: expertise_val = 1
    if expertise_val > 5: expertise_val = 5

    existing_comp = None
    for c in current_competencies:
        if c.tasktype_id == type_id:
            existing_comp = c
            break

    if existing_comp:
        if existing_comp.expertise != expertise_val:
            existing_comp.expertise = expertise_val
    else:
        new_competency = WorkerTaskType(
            worker=worker_to_edit,
            tasktype_id=type_id,
            status=HistoryStatusEnum.ACTIVE,
            expertise=expertise_val
        )
        db.session.add(new_competency)

    new_status = WorkerStatusEnum(request.form['worker_status'])
    unavailable_until_str = request.form.get("unavailable_until")

    if new_status == WorkerStatusEnum.ACTIVE:
        worker_to_edit.worker_status = WorkerStatusEnum.ACTIVE
        worker_to_edit.unavailable_until = None
    elif new_status == WorkerStatusEnum.ON_VACATION:
        if unavailable_until_str:
            worker_to_edit.worker_status = WorkerStatusEnum.ON_VACATION
            worker_to_edit.unavailable_until = datetime.strptime(unavailable_until_str, "%Y-%m-%d").date()
        else:
            flash("Для статусу 'On Vacation' необхідно вказати дату.",
                  "danger")
            return redirect(url_for('manager_views.edit_worker_profile',
                                     user_id=user_id))
    else:
        worker_to_edit.worker_status = new_status
        worker_to_edit.unavailable_until = None

    db.session.commit()
    flash(f"Дані працівника {worker_to_edit.full_name} успішно оновлено!",
          "success")
    return redirect(url_for('manager_views.manage_workers'))

except Exception as e:
    db.session.rollback()
    traceback.print_exc()
    flash(f"Сталася помилка при оновленні: {e}", "danger")

all_task_types = TaskType.query.order_by(TaskType.name).all()

```

```

                                active_competencies =
worker_to_edit.task_type_competencies.filter_by(status=HistoryStatusEnum.ACTIVE).
all()
    current_competency_map = {c.tasktype_id: c.expertise for c in
active_competencies}

    status_options = list(WorkerStatusEnum)

    return render_template(
        "manager/edit_worker_profile.html",
        worker=worker_to_edit,
        all_task_types=all_task_types,
        current_competency_map=current_competency_map,
        status_options=status_options
    )

def _calculate_priority_with_context(department_id, default_priority):
    try:
                                dept_devices_query =
db.session.query(Device.id).join(DeviceDepartment).filter(
    DeviceDepartment.department_id == department_id,
    DeviceDepartment.status == AssignmentStatusEnum.ACTIVE
)

                                worker_devices_query =
db.session.query(Device.id).join(DeviceWorker).join(Worker).join(WorkerDepartment
).filter(
    WorkerDepartment.department_id == department_id,
    WorkerDepartment.status == HistoryStatusEnum.ACTIVE,
    DeviceWorker.status == AssignmentStatusEnum.ACTIVE
)

                                all_device_ids = set(r[0] for r in
dept_devices_query.union(worker_devices_query).all())

    if not all_device_ids:
        return default_priority

    total_devices = len(all_device_ids)

    broken_count = Device.query.filter(
        Device.id.in_(all_device_ids),
        Device.status == DeviceStatusEnum.UNDER_REPAIR
    ).count()

    broken_ratio = broken_count / total_devices

    print(f"--- АНАЛІЗ ПРИОРИТЕТУ ---")
    print(f"Відділ ID: {department_id}")
    print(f"Всього пристроїв: {total_devices}, Зламаних: {broken_count}
({broken_ratio:.1%})")

    if broken_ratio > 0.6:

```

```

        print(">>> ПРИОРИТЕТ ПІДВИЩЕНО ДО КРИТИЧНОГО (5)")
        return 5

    return default_priority

except Exception as e:
    print(f"Помилка при розрахунку пріоритету: {e}")
    return default_priority

```

Файл user_views.py

Реалізація логіки користувачів-замовників

```

from flask import Blueprint, render_template, url_for, flash, redirect, request
from flask_login import login_required, current_user
from sqlalchemy.orm import joinedload
from sqlalchemy import desc

from werkzeug.security import generate_password_hash
import re, traceback, time

from . import db
from .models import (
    Worker, Task, TaskType, WorkerTask, Device, DeviceWorker,
    DeviceType, DeviceAttribute, Task_TaskType, TaskStatusEnum,
    TaskDetectionMethodEnum,
    WorkerTaskStatusEnum, DeviceStatusEnum, AssignmentStatusEnum
)
from .decorators import role_required
from .status_utils import predict_binary, predict_expert
from .worker_assign import choose_worker

user_views = Blueprint('user_views', __name__)

@user_views.route('/dashboard')
@login_required
@role_required('User')
def user_dashboard():
    my_tasks = (
        Task.query
        .filter_by(customer_id=current_user.worker_id)
        .order_by(desc(Task.created_at))
        .all()
    )
    status_options = list(TaskStatusEnum)

    return render_template(
        "user/user_dashboard.html",
        tasks=my_tasks,
        statuses=status_options,
        TaskStatusEnum=TaskStatusEnum,
        db=db,
        WorkerTask=WorkerTask
    )

```

```

)

@user_views.route('/new_request', methods=['GET', 'POST'])
@login_required
@role_required('User')
def new_request():
    if request.method == 'POST':
        request_text = request.form.get('request_text')

        if not request_text or len(request_text.strip()) < 10:
            flash('Будь ласка, опишіть вашу проблему більш детально (мінімум 10
символів).', 'warning')
            return redirect(url_for('user_views.new_request'))

        try:
            start_total_time = time.perf_counter()

            start_nlp_time = time.perf_counter()
            binary_class_id, binary_confidence = predict_binary(request_text)

            if binary_class_id == 0:
                end_total_time = time.perf_counter()

                print("--- 📊 PERFORMANCE LOG (new_request / REJECTED) 📊 ---")
                print(f"Час NLP (Воротар): {end_total_time - start_total_time:.6f}
сек")
                print(f"Результат: Відхилено як 'Не IT-запит' (Впевненість:
{binary_confidence:.2f})")
                print("-----")

                flash(f"Вашу заявку не прийнято. Система розпізнала її як 'Не IT-
запит'. (Впевненість: {binary_confidence:.2f})", "warning")
                return redirect(url_for('user_views.new_request'))

            predicted_type_id, _, probability = predict_expert(request_text)
            end_nlp_time = time.perf_counter()

            predicted_task_type = TaskType.query.get(predicted_type_id)

            if not predicted_task_type:
                flash(f"Не вдалося розпізнати тип завдання (внутрішня помилка).",
"danger")
                return redirect(url_for('user_views.new_request'))

            new_task = Task(
                text=request_text.strip(),
                priority=predicted_task_type.priority,
                customer_id=current_user.worker_id,
                status=TaskStatusEnum.NEW
            )
            db.session.add(new_task)
            db.session.flush()

```

```

new_task_type_link = Task_TaskType(
    task_id=new_task.id,
    tasktype_id=predicted_task_type.tasktype_id,
    detection_method=TaskDetectionMethodEnum.NLP_AUTO,
    probability=probability,
    assigned_by_id=None,
    is_active=True
)
db.session.add(new_task_type_link)

start_assign_time = time.perf_counter()
assignment_result = choose_worker(task=new_task)
end_assign_time = time.perf_counter()
print(assignment_result)

if assignment_result:
    new_assignment = WorkerTask(
        task_id=new_task.id,
        worker_id=assignment_result['worker_id'],
        assign_method=assignment_result['method'],
        assigned_by_id=current_user.id,
        status=WorkerTaskStatusEnum.ACTIVE
    )
    db.session.add(new_assignment)
    flash_msg = "Вашу заявку прийнято та призначено виконавця!"
    flash_category = "success"
else:
    flash_msg = "Вашу заявку прийнято, але не вдалося знайти виконавця.
Зверніться до вашого менеджера."
    flash_category = "warning"

start_commit_time = time.perf_counter()
db.session.commit()
end_commit_time = time.perf_counter()

end_total_time = time.perf_counter()

print("--- 📊 PERFORMANCE LOG (new_request / ACCEPTED) 📊 ---")
print(f"Час NLP (Воротар + Експерт): {end_nlp_time - start_nlp_time:.6f}
сек")
        print(f"Час розподілу (choose_worker): {end_assign_time -
start_assign_time:.6f} сек")
        print(f"Час фіксації (db.commit): {end_commit_time -
start_commit_time:.6f} сек")
        print(f"---")
    print(f"ЗАГАЛЬНИЙ ЧАС ОБРОБКИ: {end_total_time - start_total_time:.6f}
сек")
    print("-----")

    flash(flash_msg, flash_category)
    return redirect(url_for('user_views.user_dashboard'))

except Exception as e:

```

```

        db.session.rollback()
        traceback.print_exc()
        flash(f"Сталася непередбачувана помилка при створенні заявки: {e}",
            "danger")

    return render_template("user/new_request.html")

@user_views.route('/new_personal_device_request', methods=['GET', 'POST'])
@login_required
@role_required('User')
def new_personal_device_request():
    if request.method == 'POST':
        try:
            form_data = request.form
            device_type_id = form_data.get('device_type_id')
            justification = form_data.get('justification')

            if not device_type_id or not justification:
                flash("Будь ласка, оберіть тип пристрою та вкажіть обґрунтування.",
                    "danger")
                return redirect(url_for('user_views.new_personal_device_request'))

            device_type = DeviceType.query.get(device_type_id)
            request_task_type = TaskType.query.filter_by(name='Device
Management').first()
            if not request_task_type:
                flash("Критична помилка: тип завдання 'Device Request' не знайдено.",
                    "danger")
                return redirect(url_for('user_views.new_personal_device_request'))

            desired_specs = []
            for key, value in form_data.items():
                if key.startswith('attr_') and value.strip():
                    attribute_id = int(key.split('_')[1])
                    attribute = DeviceAttribute.query.get(attribute_id)
                    if attribute:
                        desired_specs.append(f"- {attribute.name}: {value.strip()}")

            specs_text = "\n\n--- Бажані характеристики ---\n" +
"\n".join(desired_specs) if desired_specs else ""
            task_text = (
                f"Запит на новий ОСОБИСТИЙ пристрій.\n"
                f"Тип пристрою: {device_type.name}.\n"
                f"Призначення: для працівника ID:{current_user.id}
({current_user.full_name}).\n\n"
                f"Обґрунтування: {justification}"
                f"{specs_text}"
            )

            new_task = Task(text=task_text, priority=request_task_type.priority,
customer_id=current_user.id)
            db.session.add(new_task)
            db.session.flush()

```

```

new_task_type_link = Task_TaskType(
    task_id=new_task.id,
    tasktype_id=request_task_type.tasktype_id,
    detection_method=TaskDetectionMethodEnum.MANUAL_OVERRIDE,
    assigned_by_id=current_user.id
)
db.session.add(new_task_type_link)

assignment_result = choose_worker(task=new_task)
flash_msg = "Заявку на новий пристрій успішно створено!"
flash_category = "success"
if assignment_result:
    new_assignment = WorkerTask(
        task_id=new_task.id,
        worker_id=assignment_result['worker_id'],
        assign_method=assignment_result['method'],
        assigned_by_id=current_user.id,
        status=WorkerTaskStatusEnum.ACTIVE
    )
    db.session.add(new_assignment)
    flash_msg += " Виконавця призначено."
else:
    flash_msg += " Не вдалося знайти виконавця. Зверніться до IT-
відділу."
    flash_category = "warning"

db.session.commit()
flash(flash_msg, flash_category)
return redirect(url_for('user_views.user_dashboard'))

except Exception as e:
    db.session.rollback()
    flash(f"Сталася помилка при створенні заявки: {e}", "danger")
    traceback.print_exc()

available_device_types = DeviceType.query.order_by(DeviceType.name).all()
return render_template("user/new_personal_device_request.html",
device_types=available_device_types)

```

Файл worker_assign.py

Реалізація логіки визначення та призначення виконавця завдання

```

import random
from sqlalchemy import func

from . import db
from .models import (
    Worker, Role, WorkerTaskType, WorkerTask,
    WorkerStatusEnum, HistoryStatusEnum, WorkerTaskStatusEnum, WorkerRole, Task,
    WorkerTaskAssignmentMethodEnum, WorkerDepartment, Department
)

```

```

def choose_worker(task: Task, ignore_max_load: bool = False, exclude_worker_id:
int = None):

    task_type = task.active_task_type
    if not task_type:
        print(f"ПОМИЛКА: Не вдалося визначити активний тип для завдання #{task.id}")
        return None

    task_priority = task_type.priority

    worker_role_obj = Role.query.filter_by(name='Worker').first()
    if not worker_role_obj: return None

    query = (
        Worker.query
        .join(Worker.roles_history)
        .filter(WorkerRole.role_id == worker_role_obj.role_id, WorkerRole.status
== HistoryStatusEnum.ACTIVE)
        .join(Worker.task_type_competencies)
        .filter(WorkerTaskType.tasktype_id == task_type.tasktype_id,
WorkerTaskType.status == HistoryStatusEnum.ACTIVE)
        .filter(Worker.worker_status == WorkerStatusEnum.ACTIVE)
    )

    if exclude_worker_id:
        query = query.filter(Worker.worker_id != exclude_worker_id)

    eligible_workers = query.all()

    candidates = []
    if eligible_workers:
        for worker in eligible_workers:
            current_load = db.session.query(func.count(WorkerTask.id)).filter(
                WorkerTask.worker_id == worker.worker_id,
                WorkerTask.status == WorkerTaskStatusEnum.ACTIVE
            ).scalar() or 0

            is_overloaded = worker.max_load is not None and current_load >=
worker.max_load
            if is_overloaded and task_priority < 3 and not ignore_max_load:
                continue

            competency =
worker.task_type_competencies.filter_by(tasktype_id=task_type.tasktype_id,
status=HistoryStatusEnum.ACTIVE).first()
            expertise = competency.expertise if competency and competency.expertise
is not None else 1

            score = (expertise * 10) - current_load
            candidates.append((worker, score))

    if candidates:
        max_score = max(score for _, score in candidates)
        top_candidates = [w for w, s in candidates if s == max_score]

```

```

        chosen_worker = random.choice(top_candidates)
        return {
            'worker_id': chosen_worker.worker_id,
            'method': WorkerTaskAssignmentMethodEnum.AUTOMATIC
        }
    else:
        print(f"Кандидатів-виконавців не знайдено для завдання #{task.id}. Спроба ескалації на IT-менеджера.")

        it_department =
        Department.query.filter(Department.name.ilike('%IT%')).first()
        manager_role = Role.query.filter_by(name='Manager').first()

        if not it_department or not manager_role:
            print(f"КРИТИЧНА ПОМИЛКА: Не знайдено IT відділ ('%IT%') або роль 'Manager'. Неможливо ескалувати завдання #{task.id}")
            return None

        it_manager = (
            Worker.query
            .join(WorkerDepartment, Worker.worker_id == WorkerDepartment.worker_id)
            .join(WorkerRole, Worker.worker_id == WorkerRole.worker_id)
            .filter(
                WorkerDepartment.department_id == it_department.department_id,
                WorkerDepartment.status == HistoryStatusEnum.ACTIVE,
                WorkerRole.role_id == manager_role.role_id,
                WorkerRole.status == HistoryStatusEnum.ACTIVE,
                Worker.worker_status == WorkerStatusEnum.ACTIVE
            )
            .first()
        )

        if it_manager:
            print(f"Ескалація завдання #{task.id} до IT-менеджера ID:{it_manager.worker_id} ({it_manager.full_name})")
            return {
                'worker_id': it_manager.worker_id,
                'method': WorkerTaskAssignmentMethodEnum.ESCALATED_TO_IT_MANAGER
            }
        else:
            print(f"КРИТИЧНА ПОМИЛКА: Не знайдено активного IT-менеджера у відділі '{it_department.name}' для ескалації завдання #{task.id}")
            return None

```

Файл worker_views.py

Реалізація логіки працівника-виконавця

```

from flask import Blueprint, render_template, url_for
from flask_login import login_required, current_user
from sqlalchemy.orm import joinedload
from sqlalchemy import desc

```

```

from flask import Blueprint, render_template, request, flash, redirect, url_for,
jsonify
from datetime import datetime
import traceback
import re
from .worker_assign import choose_worker
from werkzeug.security import generate_password_hash

from . import db
from .models import (WorkerTask, Task, Worker, WorkerDepartment, TaskType,
Task_TaskType, TaskStatusEnum, WorkerTaskStatusEnum, Device, DeviceType,
DeviceStatusEnum, DeviceWorker, DeviceDepartment, AssignmentStatusEnum,
TaskDetectionMethodEnum, WorkerTaskAssignmentMethodEnum)
from .decorators import role_required

worker_views = Blueprint('worker_views', __name__)

@worker_views.route('/dashboard')
@login_required
@role_required('Worker')
def worker_dashboard():
    """
    Відображає список усіх завдань, коли-небудь призначених поточному працівнику,
    включаючи активні, завершені та перепризначені.
    """
    my_assignments = (
        WorkerTask.query
        .filter_by(worker_id=current_user.worker_id)
        .options(
            joinedload(WorkerTask.task)
            .joinedload(Task.customer)
        )
        .order_by(desc(WorkerTask.assigned_at))
        .all()
    )

    status_options = list(WorkerTaskStatusEnum)

    return render_template(
        "worker/worker_dashboard.html",
        assignments=my_assignments,
        statuses=status_options
    )

@worker_views.route('/task/<int:task_id>', methods=['GET', 'POST'])
@login_required
@role_required('Worker')
def view_task(task_id):
    task = Task.query.get_or_404(task_id)
    assignment = task.current_assignment

    task_type_name = task.active_task_type.name if task.active_task_type else None

```

```

if task_type_name == 'Device Management':
    if request.method == 'POST':
        try:
            device_id_to_assign = int(request.form.get('device_id'))
            target_info_str = request.form.get('target_info')

            device = Device.query.get(device_id_to_assign)
            if not device:
                flash("Обраний пристрій не знайдено (можливо, його видалили).",
                    "danger")
                return redirect(url_for('worker_views.view_task',
                    task_id=task_id))

            if device.status != DeviceStatusEnum.AVAILABLE:
                flash("Обраний пристрій вже недоступний (його статус змінився).
Оновіть сторінку.", "danger")
                return redirect(url_for('worker_views.view_task',
                    task_id=task_id))

            if target_info_str.startswith('worker:'):
                try:
                    worker_id = int(target_info_str.split(':')[1])
                    target_worker = Worker.query.get(worker_id)
                    if not target_worker:
                        raise ValueError(f"Працівника з ID {worker_id} не
                            знайдено.")

                    new_device_assignment = DeviceWorker(
                        device_id=device.id,
                        worker_id=worker_id,
                        status=AssignmentStatusEnum.ACTIVE
                    )
                    db.session.add(new_device_assignment)
                    assignment_target_msg = f"працівнику
{target_worker.full_name}"
                except (ValueError, IndexError) as e:
                    raise ValueError(f"Некоректний формат цілі призначення для
працівника: {target_info_str}. Помилка: {e}")

            elif target_info_str == 'department':
                department_id = task.customer.current_department.department_id
                if not department_id:
                    raise ValueError(f"Не вдалося визначити відділ для
призначення пристрою (замовник ID: {task.customer_id}).")

                new_device_assignment = DeviceDepartment(
                    device_id=device.id,
                    department_id=department_id,
                    status=AssignmentStatusEnum.ACTIVE
                )
                db.session.add(new_device_assignment)
                assignment_target_msg = f"відділу
{task.customer.current_department.name}"
            else:

```

```

        raise ValueError(f"Невідомий тип цілі призначення:
{target_info_str}")

        device.status = DeviceStatusEnum.IN_USE

        task.status = TaskStatusEnum.CLOSED
        task.finished_at = datetime.utcnow()

        assignment.status = WorkerTaskStatusEnum.FINISHED
        assignment.finished_at = datetime.utcnow()

        db.session.commit()
        traceback.print_exc()
        flash(f"Пристрій '{device.name}' успішно призначено!", "success")
        return redirect(url_for('worker_views.worker_dashboard'))

    except Exception as e:
        db.session.rollback()
        flash(f"Сталася помилка при призначенні пристрою: {e}", "danger")
        traceback.print_exc()

    desired_specs = _parse_desired_specs(task.text)

    device_type_name_match = re.search(r"Тип пристрою: (.*)\.", task.text)
    if not device_type_name_match:
        flash("Не вдалося визначити тип пристрою з тексту заявки.", "warning")
        return render_template('worker/task_device_request.html', task=task,
assignment=assignment, recommendations=[])

    device_type = DeviceType.query.filter_by(name=device_type_name_match.group(1)).first()
    print(device_type.name)

    recommendations = []
    if device_type:
        available_devices = Device.query.filter_by(
            device_type_id=device_type.id,
            status=DeviceStatusEnum.AVAILABLE
        ).all()

        for device in available_devices:
            score = 0
            actual_specs = device.specs_dict
            for spec_name, desired_value in desired_specs.items():
                if str(actual_specs.get(spec_name, '')).strip() ==
str(desired_value).strip():
                    score += 1

            recommendations.append({'device': device, 'score': score})

        recommendations.sort(key=lambda x: (-x['score'], x['device'].name))

    target_info = _parse_assignment_target(task.text)
    print(target_info)

```

```

        target_info_str = f"worker:{target_info['id']}" if target_info.get('type')
        == 'worker' else 'department'
        print(target_info_str)

        return render_template(
            'worker/task_device_request.html',
            task=task,
            assignment=assignment,
            recommendations=recommendations,
            total_score=len(desired_specs),
            target_info_str=target_info_str
        )
    elif task_type_name == 'Device Return':
        device_to_return = task.device
        if not device_to_return:
            flash("Помилка: Не вдалося знайти пристрій, пов'язаний з цим
завданням.", "danger")
            return redirect(url_for('worker_views.worker_dashboard'))

        if request.method == 'POST':
            try:
                new_status_str = request.form.get('new_status')
                if new_status_str == 'available':
                    final_device_status = DeviceStatusEnum.AVAILABLE
                elif new_status_str == 'under_repair':
                    final_device_status = DeviceStatusEnum.UNDER_REPAIR
                else:
                    raise ValueError("Некоректний статус обрано.")

                device_to_return.status = final_device_status

                device_worker_link =
DeviceWorker.query.filter_by(device_id=device_to_return.id,
status=AssignmentStatusEnum.ACTIVE).first()
                device_department_link =
DeviceDepartment.query.filter_by(device_id=device_to_return.id,
status=AssignmentStatusEnum.ACTIVE).first()

                if device_worker_link:
                    device_worker_link.status = AssignmentStatusEnum.RETURNED
                    device_worker_link.returned_at = datetime.utcnow()
                elif device_department_link:
                    device_department_link.status = AssignmentStatusEnum.RETURNED
                    device_department_link.returned_at = datetime.utcnow()

                task.status = TaskStatusEnum.CLOSED
                task.finished_at = datetime.utcnow()

                assignment.status = WorkerTaskStatusEnum.FINISHED
                assignment.finished_at = datetime.utcnow()
                assignment.comment = f"Пристрій отримано. Новий статус:
{final_device_status.value}."

```

```

        if final_device_status == DeviceStatusEnum.UNDER_REPAIR:
            repair_task_type = TaskType.query.filter_by(name='Device
Repair').first()
            if repair_task_type:
                repair_task_text = (
                    f"Завдання на ремонт пристрою.\n"
                    f"Пристрій: {device_to_return.name} (SN:
{device_to_return.serial_number})\n"
                    f"Попередній власник/відділ: {task.customer.full_name
if task.customer else 'Невідомо'}\n"
                    f"Виявлено потребу в ремонті працівником:
{current_user.full_name}"
                )
                new_repair_task = Task(
                    text=repair_task_text,
                    priority=repair_task_type.priority,
                    customer_id=current_user.id,
                    device_id=device_to_return.id
                )
                db.session.add(new_repair_task)
                db.session.flush()

                repair_task_type_link = Task_TaskType(
                    task_id=new_repair_task.id,
                    tasktype_id=repair_task_type.tasktype_id,
                    detection_method=TaskDetectionMethodEnum.MANUAL_OVERR
IDE,
                    assigned_by_id=current_user.id
                )
                db.session.add(repair_task_type_link)

                repair_assignment_result =
choose_worker(task=new_repair_task)
                if repair_assignment_result:
                    new_repair_assignment = WorkerTask(
                        task_id=new_repair_task.id,
                        worker_id=repair_assignment_result['worker_id'],
                        assign_method=repair_assignment_result['method'],
                        assigned_by_id=current_user.id,
                        status=WorkerTaskStatusEnum.ACTIVE
                    )
                    db.session.add(new_repair_assignment)
                    repair_task_created_msg = "Створено нове завдання на
ремонт та призначено виконавця."
                else:
                    repair_task_created_msg = "Створено нове завдання на
ремонт, але не вдалося знайти виконавця."
                else:
                    repair_task_created_msg = "ПОМИЛКА: Тип завдання 'Device
Repair' не знайдено, завдання на ремонт не створено!"

                db.session.commit()

```

```

        flash(f"Пристрій '{device_to_return.name}' успішно опрацьовано.
Новий статус: {final_device_status.value}.", "success")
        return redirect(url_for('worker_views.worker_dashboard'))

    except Exception as e:
        db.session.rollback()
        flash(f"Сталася помилка при опрацюванні повернення: {e}", "danger")
        traceback.print_exc()

    return render_template(
        'worker/task_device_return.html',
        task=task,
        assignment=assignment,
        device=device_to_return
    )
elif task_type_name == 'Device Repair':
    device_to_repair = task.device
    if not device_to_repair:
        flash("Помилка: Не вдалося знайти пристрій, пов'язаний з цим завданням
на ремонт.", "danger")
        return redirect(url_for('worker_views.worker_dashboard'))

if request.method == 'POST':
    try:
        repair_outcome = request.form.get('repair_outcome')
        comment = request.form.get('comment')

        if repair_outcome == 'fixed':
            final_device_status = DeviceStatusEnum.AVAILABLE
        elif repair_outcome == 'decommissioned':
            final_device_status = DeviceStatusEnum.DECOMMISSIONED
        else:
            raise ValueError("Некоректний результат ремонту обрано.")

        device_to_repair.status = final_device_status

        device_worker_link = DeviceWorker.query.filter_by(
            device_id=device_to_repair.id,
            status=AssignmentStatusEnum.ACTIVE
        ).first()

        device_department_link = DeviceDepartment.query.filter_by(
            device_id=device_to_repair.id,
            status=AssignmentStatusEnum.ACTIVE
        ).first()

        assignment_closed_msg = ""
        if device_worker_link:
            device_worker_link.status = AssignmentStatusEnum.RETURNED
            device_worker_link.returned_at = datetime.utcnow()
            assignment_closed_msg = "Попереднє призначення працівнику
закрито."

        print(f"Деактивовано DeviceWorker ID: {device_worker_link.id}")
        elif device_department_link:

```

```

        device_department_link.status = AssignmentStatusEnum.RETURNED
        device_department_link.returned_at = datetime.utcnow()
        assignment_closed_msg = "Попереднє призначення відділу закрито."
        print(f"Деактивовано DeviceDepartment ID:
{device_department_link.id}")

        task.status = TaskStatusEnum.CLOSED
        task.finished_at = datetime.utcnow()

        assignment.status = WorkerTaskStatusEnum.FINISHED
        assignment.finished_at = datetime.utcnow()
        assignment.comment = f"Ремонт завершено. Результат:
{final_device_status.value}. {comment or ''}".strip()

        db.session.commit()
        flash(f"Ремонт пристрою '{device_to_repair.name}' завершено. Новий
статус: {final_device_status.value}.", "success")
        return redirect(url_for('worker_views.worker_dashboard'))

    except Exception as e:
        db.session.rollback()
        flash(f"Сталася помилка при завершенні ремонту: {e}", "danger")
        traceback.print_exc()

    return render_template(
        'worker/task_device_repair.html',
        task=task,
        assignment=assignment,
        device=device_to_repair
    )
else:
    if request.method == 'POST':
        try:
            new_status_value = request.form.get('assignment_status')
            comment = request.form.get('comment')

            if not new_status_value:
                flash("Необхідно обрати новий статус.", "warning")
                return redirect(url_for('worker_views.view_task',
task_id=task_id))

            new_status = WorkerTaskStatusEnum(new_status_value)

            assignment.status = new_status
            assignment.comment = comment

            if new_status == WorkerTaskStatusEnum.FINISHED:
                assignment.finished_at = datetime.utcnow()
                assignment.task.status = TaskStatusEnum.CLOSED

            db.session.commit()
            flash("Статус завдання успішно оновлено!", "success")
            return redirect(url_for('worker_views.worker_dashboard'))

```

```
except Exception as e:
    db.session.rollback()
    flash(f"Сталася помилка при оновленні статусу: {e}", "danger")
    traceback.print_exc()

available_statuses = [
    WorkerTaskStatusEnum.ACTIVE,
    WorkerTaskStatusEnum.FINISHED
]

return render_template(
    'worker/task_generic.html',
    task=task,
    assignment=assignment,
    available_statuses=available_statuses,
    WorkerTaskStatusEnum = WorkerTaskStatusEnum
)
```

ДОДАТОК Д

Результати експериментів

№ Завдання	Час NLP (сек)	Час Розподілу (сек)	Час Фіксації (сек)	Загальний Час (сек)
1	0.712579	0.044748	0.008934	0.798177
2	0.085059	0.012268	0.002856	0.104121
3	0.079454	0.011364	0.003503	0.098887
4	0.081573	0.035086	0.002905	0.124431
5	0.085845	0.020641	0.002793	0.113469
6	0.074810	0.009965	0.002574	0.091566
7	0.078635	0.014062	0.002729	0.100202
8	0.085448	0.015117	0.002391	0.107406
9	0.079733	0.019998	0.002448	0.106929
10	0.076438	0.022281	0.002401	0.105710
11	0.087949	0.015182	0.002320	0.109421
12	0.070782	0.023614	0.002761	0.101636
13	0.082554	0.026884	0.006224	0.120243
14	0.087730	0.014537	0.002541	0.111565
15	0.084513	0.010783	0.002632	0.102120
16	0.086944	0.012472	0.002442	0.105712
17	0.119146	0.016259	0.008596	0.154426
18	0.076836	0.013357	0.003623	0.099703
19	0.087020	0.013864	0.002402	0.103286
20	0.085954	0.015979	0.005584	0.107517
21	0.106572	0.032280	0.003905	0.142757
22	0.089745	0.030380	0.005887	0.126012
23	0.097456	0.028520	0.008361	0.134337
24	0.077385	0.010282	0.003199	0.090866
25	0.101825	0.014353	0.005798	0.121976
26	0.103859	0.034540	0.003548	0.141947
27	0.089906	0.012508	0.006789	0.109203
28	0.084279	0.032148	0.004192	0.120619
29	0.098875	0.019579	0.003122	0.121576
30	0.073573	0.034450	0.004233	0.112256
31	0.078004	0.017210	0.005344	0.100558
32	0.087564	0.012876	0.003722	0.104162
33	0.086041	0.012524	0.005081	0.103646
34	0.101098	0.018947	0.007308	0.127353
35	0.092105	0.014546	0.008523	0.115174

36	0.118011	0.020694	0.006285	0.144990
37	0.085323	0.034504	0.002546	0.122373
38	0.118519	0.019053	0.005677	0.143249
39	0.084156	0.028377	0.002391	0.114924
40	0.118208	0.018258	0.006214	0.142680
41	0.083956	0.024987	0.002554	0.111497
42	0.097935	0.030995	0.004005	0.132935
43	0.075711	0.011524	0.003100	0.090335
44	0.084516	0.022710	0.004726	0.111952
45	0.091227	0.016053	0.007396	0.114676
46	0.105542	0.032168	0.006742	0.144452
47	0.079376	0.028604	0.005557	0.113537
48	0.105035	0.027817	0.006701	0.139553
49	0.114501	0.034131	0.002658	0.151290
50	0.110447	0.032892	0.002907	0.146246

ДОДАТОК Е

Результати перевірки роботи на співпадіння

Дата звіту12/4/2025
Дата редагування12/5/2025

Документ прийнятий

Звіт подібності

Метадані

Назва організації
National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute
Заголовок
ІП-43мп_Цуканова_ПЗ
АвторНауковий керівник / Експерт
ЦукановаЛіхоузова Т.А.
підрозділ
ФІОТ, К-а інформатики та програмної інженерії

Обсяг знайдених подібностей

Коефіцієнт подібності визначає, який відсоток тексту по відношенню до загального обсягу тексту було знайдено в різних джерелах. Зверніть увагу, що високі значення коефіцієнта не автоматично означають плагіат. Звіт має аналізувати компетентна / уповноважена особа.

1.57%

1.57%

КП 1

7690

Кількість слів

61116

Кількість символів

Тривога

У цьому розділі ви знайдете інформацію щодо текстових спотворень. Ці спотворення в тексті можуть говорити про **МОЖЛИВІ** маніпуляції в тексті. Спотворення в тексті можуть мати навімисний характер, але частіше характер технічних помилок при конвертації документа та його збереженні, тому ми рекомендуємо вам підходити до аналізу цього модуля відповідально. У разі виникнення запитань, просимо звертатися до нашої служби підтримки.

Заміна букв		2
Інтервали		0
Мікропробіли		0
Білі знаки		0
Парафрази (SmartMarks)		3

Джерела

Нижче наведений список джерел. В цьому списку є джерела із різних баз даних. Колір тексту означає в якому джерелі він був знайдений. Ці джерела і значення Коефіцієнту Подібності не відображають прямого плагіату. Необхідно відкрити кожне джерело і проаналізувати зміст і правильність оформлення джерела.

10 найдовших фраз

Колір тексту

ПОРЯДКОВИЙ НОМЕР	НАЗВА ТА АДРЕСА ДЖЕРЕЛА URL (НАЗВА БАЗИ)	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
1	Програмне та математичне забезпечення для генерації діалогових систем з підтримкою української мови 3/15/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	33 0.43 %
2	https://softtech.kpi.ua/	17 0.22 %

3	Програмне забезпечення обробки природної мови для потоків текстових даних в режимі реального часу [REDACTED] 3/15/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	12 0.16 %
4	ІП-32мп_Галайко_ПЗ [REDACTED] 12/2/2024 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	11 0.14 %
5	Програмне забезпечення обробки природної мови для потоків текстових даних в режимі реального часу [REDACTED] 3/15/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	8 0.10 %
6	https://openarchive.nure.ua/bitstreams/320eb302-16da-4292-8a2a-3e85e8a24bd8/download	8 0.10 %
7	Інтелектуальна система розміщення рекламного контенту [REDACTED] 3/15/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	8 0.10 %
8	https://softtech.kpi.ua/	8 0.10 %
9	Програмне та математичне забезпечення для генерації діалогових систем з підтримкою української мови [REDACTED] 3/15/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	6 0.08 %
10	https://openarchive.nure.ua/bitstreams/320eb302-16da-4292-8a2a-3e85e8a24bd8/download	5 0.07 %
з домашньої бази даних (1.01 %)		
ПОРЯДКОВИЙ НОМЕР	ЗАГОЛОВОК	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
1	Програмне та математичне забезпечення для генерації діалогових систем з підтримкою української мови [REDACTED] 3/15/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	39 (2) 0.51 %
2	Програмне забезпечення обробки природної мови для потоків текстових даних в режимі реального часу [REDACTED] 3/15/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	20 (2) 0.26 %
3	ІП-32мп_Галайко_ПЗ [REDACTED] 12/2/2024 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	11 (1) 0.14 %
4	Інтелектуальна система розміщення рекламного контенту [REDACTED] 3/15/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	8 (1) 0.10 %
з програми обміну базами даних (0.07 %)		
ПОРЯДКОВИЙ НОМЕР	ЗАГОЛОВОК	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)

5	БКР_Програмна система відстеження гуманітарних вантажів у логістичних ланцюгах постачання Галашевський 6/1/2025 Khmelnitskyi National University (Кафедра інженерії програмного забезпечення)	5 (1) 0.07 %
з Інтернету (0.49 %)		
ПОРЯДКОВИЙ НОМЕР	ДЖЕРЕЛО URL	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
6	https://softtech.kpi.ua/	25 (2) 0.33 %
7	https://openarchive.nure.ua/bitstreams/320eb302-16da-4292-8a2a-3e85e8a24bd8/download	13 (2) 0.17 %
Список прийнятих фрагментів (немає прийнятих фрагментів)		
ПОРЯДКОВИЙ НОМЕР	ЗМІСТ	КІЛЬКІСТЬ ОДНАКОВИХ СЛІВ (ФРАГМЕНТІВ)